

x86-64 Assembly Language Programming with Ubuntu-2

남궁명수

Chapter8. 주소 지정 방식(Addressing Modes).....	4
[8-1] 주소와 값(Addressing and Values).....	4
[8-1-1] 레지스터 모드 주소 지정(Register Mode Addressing)	5
[8-1-2] 즉시값 모드 주소 지정(Immediate Mode Addressing).....	5
[8-1-3] 메모리 모드 주소 지정(Memory Mode Addressing)	5
[8-2] 예제 프로그램, 리스트 합계(Example Program, List Summation)	8
[8-2] 예제 프로그램, 피라미드의 면적과 부피	10
Chapter9. 프로세스 스택(Process Stack).....	15
[9-1] Stack Example.....	15
[9-2] 스택 명령어(Stack Instructions).....	16
[9-2-1] push <op64>	17
[9-2-2] pop <op64>.....	17
[9-3] 스택 구현(Stack Implementation)	18
[9-3-1] 스택 구조(Stack Layout).....	18
[9-3-2] 스택 연산 (Stack Operations).....	19
[9-4] 스택 예제	21
Chapter10. 프로그램 개발(Program Development)	23
[10-1] 문제 이해.....	23
[10-2] 알고리즘 작성.....	24
[10-3] 프로그램 구현.....	26
[10-4] 프로그램 테스트/디버깅.....	28
[10-5] 오류 용어.....	29
[10-5-1] Assembler Error(어셈블러 오류)	29
[10-5-2] Run-time Error(실행시간 오류).....	30
[10-5-3] Logic Error(논리 오류)	30
Chapter11. 매크로(Macros)	31
[11-1] 단일 행 매크로(Single-Line Macros).....	31
[11-2] 다중 행 매크로(Multi-Line Macros)	32
[11-2-1] 매크로 정의(Macro Definition).....	32
[11-2-2] 매크로 사용(Using a Macro).....	33
[11-3] 매크로 예제(Macro Example).....	34
[11-4] 매크로 디버깅(Debugging Macros).....	35
Chapter12. 함수(Functions)	36

[12-1] 업데이트된 링크 명령.....	36
[12-2] 디버깅 명령	37
[12-2-1] 디버거 명령 next.....	37
[12-2-1] 디버거 명령 step.....	37
[12-3] 스택 동적 지역 변수	38
[12-4] 함수 선언.....	38
[12-5] 표준 호출 규약.....	39
[12-5-1] System V AMD64 ABI.....	39
[12-6] 링키지(Linkage)	39
[12-6-1] call 명령.....	39
[12-6-2] ret 명령	40
[12-6-3] 스택 손상에 대한 주의.....	40
[12-6-4] 함수 호출 요약	40
[12-7] 인수 전달(Argument Transmission).....	40
[12-7-1] call-by-value	41
[12-7-2] call-by-reference.....	41
[12-7-3] 인수 전달 방법	41
[12-8] System V AMD64 ABI 에서의 실제 인수 전달	44
[12-8-1] 정수형 인수 전달 레지스터	44
[12-8-2] 7 번째 인수부터는 스택 사용.....	45
[12-8-3] 반환값(Return Value)	45
[12-8-4] call-by-reference 의 실제 구현.....	46
[12-8-5] 부동소수점 인수	46
[12-8-6] Caller-Saved vs Callee-Saved 레지스터	46
[12-8-7] 전체 흐름 정리	47
[12-8-8] 전체 구조.....	48
[12-9] 호출 규약(Calling Convention)	48
[12-9-1] 매개변수 전달(Parameter Passing)	49
[12-9-2] 레지스터 사용(Register Usage).....	51
[12-9-3] 콜 프레임(Call Frame).....	53
[12-9-4] Red Zone.....	55
[12-10] 예제: 통계함수 1 - 리프 함수.....	57
[12-10-1] 고급 언어(HLL) 호출 형태	57
[12-10-2] Caller(호출자).....	57
[12-10-3] Callee(피호출자)	59

[12-11] 예제: 통제함수 2 - 리프 함수.....	61
[12-11-1] Caller(호출자).....	62
[12-11-2] Callee(피호출자).....	63
[12-12] 스택 기반 지역 변수(Stack-Based Local Variables).....	65
[12-13] 요약 - 표준 호출 규약 정리.....	69
[12-13-1] 호출자 동작.....	69
[12-13-2] 피호출자 동작.....	70
Chapter13. 시스템 서비스(System Services).....	73
[13-1] 시스템 서비스 호출(Calling System Services).....	73
[13-2] 개행 문자.....	75
[13-3] 콘솔 문자(Console Output).....	76
[13-3-1] 예제: Console Output.....	78
[13-4] 콘솔 입력(Console Input).....	83
[13-4-1] 예제: Console Input.....	85
[13-5] 파일 열기 작업(File Open Operations).....	90
[13-5-1] 파일 열기(File Open).....	91
[13-5-2] 파일 열기/생성(File Open/Create).....	92
[13-6] 파일 읽기(File Read).....	93
[13-7] 파일 쓰기(File Write).....	94
[13-8] 파일 작업 예제(File Operations Examples).....	94
[13-8-1] 예제: 파일 쓰기.....	94
[13-8-1] 예제: 파일 읽기.....	103

Chapter8. 주소 지정 방식(Addressing Modes)

이 장은 x86-64 아키텍처에서의 주소 지정 방식과 관련된 주소 조작에 대한 기본적인 정보를 제공한다.

주소 지정 방식(addressing modes)은 메모리에서 값을 접근(읽기 또는 쓰기)할 때 데이터 항목의 주소를 이용하여 접근하는 지원 방법이다.

이는 변수의 이름이나 배열의 특정 위치를 포함할 수 있다.
기본 주소 지정 방식은 다음과 같다.

- Register
- Immediate
- Memory

각 방식은 다음 절에서 예제와 함께 설명된다.
또한 배열 접근에 대한 간단한 예제가 제시된다.

[8-1] 주소와 값(Addressing and Values)

64 비트 아키텍처에서 주소는 64 비트가 필요하다.
이전 장에서 언급했듯이,
메모리에 접근하는 유일한 방법은 대괄호([])를 사용하는 것이다.

대괄호를 생략하면 메모리에 접근하지 않고 대신 그 항목의 주소를 얻는다.
예:

```
mov rax, qword [var1] ; var1 의 값을 rax 에 저장  
mov rax, var1         ; var1 의 주소를 rax 에 저장
```

대괄호를 생략해도 오류는 아니므로
어셈블러는 오류나 경고 메시지를 생성하지 않는다.

메모리에 접근할 때, 대부분의 경우 피연산자 크기는 명확하다.
예:

```
mov eax, [rbx]
```

이 명령은 메모리에서 더블워드(32 비트)를 이동한다.
그러나 일부 명령에서는 크기가 모호할 수 있다.

예:

```
inc [rbx] ; error
```

이 경우 접근되는 메모리가 바이트인지, 워드인지, 더블워드인지 명확하지 않다.
이 경우, byte, word, dword, qword 와 같은 크기 지정자가 필요하다.

예:

```
inc byte [rbx]
inc word [rbx]
inc dword [rbx]
```

각 명령은 명확하고 합법적인 명령이 되기 위해 크기 지정이 필요하다.

[8-1-1] 레지스터 모드 주소 지정(Register Mode Addressing)

레지스터 모드 주소 지정이란 피연산자가 CPU 레지스터(eax, ebx 등)인 경우이다.

예:

```
mov eax, ebx
```

eax 와 ebx 는 모두 레지스터 모드 주소 지정이다.

[8-1-2] 즉시값 모드 주소 지정(Immediate Mode Addressing)

즉시값 모드 주소 지정이란 피연산자가 즉시값인 경우이다.

예:

```
mov eax, 123
```

목적지 피연산자 eax 는 레지스터 모드 주소 지정이다.

123 은 즉시값 모드 주소 지정이다.

이 예에서 목적지 피연산자는 즉시값 모드가 될 수 없다는 것은 분명하다.

[8-1-3] 메모리 모드 주소 지정(Memory Mode Addressing)

메모리 모드 주소 지정이란 피연산자가 메모리 위치(주소를 통해 접근됨)인 경우이다.

이를 간접 참조(indirection) 또는 역참조(dereferencing)라고 한다.

가장 기본적인 메모리 모드 주소 지정은 이전 장에서 많이 사용되었다.

예:

```
mov rax, qword [qNum]
```

이 명령은 변수 qNum 의 메모리 위치에 접근하여 저장된 값을 가져온다.

이 경우 CPU 는 값을 가져올 때까지 기다려야 하므로

즉시값을 사용하는 유사한 연산보다 약간 더 오래 걸릴 수 있다.

배열을 접근할 때는 더 일반화된 방법이 필요하다.

특히, 주소를 레지스터에 저장하고

변수 이름 대신 레지스터를 이용하여 간접 참조를 수행할 수 있다.

예를 들어, 다음 선언이 있다고 가정하자.

```
lst dd 101, 103, 105, 107
```

십진수 101 은 16 진수로 0x00000065

메모리 배치는 다음과 같다.

Value	Address	Offset	Index
00	0x00000000006000ef	lst + 15	
00	0x00000000006000ee	lst + 14	
00	0x00000000006000ed	lst + 13	
6b	0x00000000006000ec	lst + 12	lst[3]
00	0x00000000006000eb	lst + 11	
00	0x00000000006000ea	lst + 10	
00	0x00000000006000e9	lst + 9	
69	0x00000000006000e8	lst + 8	lst[2]
00	0x00000000006000e7	lst + 7	
00	0x00000000006000e6	lst + 6	
00	0x00000000006000e5	lst + 5	
67	0x00000000006000e4	lst + 4	lst[1]
00	0x00000000006000e3	lst + 3	
00	0x00000000006000e2	lst + 2	
00	0x00000000006000e1	lst + 1	
65	0x00000000006000e0	lst + 0	lst[0]

첫 번째 배열 요소는 다음과 같이 접근할 수 있다.

```
mov eax, dword [lst]
```

또 다른 방법:

```
mov rbx, lst  
mov eax, dword [rbx]
```

이 예에서 리스트의 시작 주소(기준 주소, base address)를 rbx 에 저장한 후,
해당 주소의 값을 rax 에 저장한다.

이 방식은 배열의 다른 요소에 쉽게 접근할 수 있게 한다.

메모리는 “바이트 주소 지정(byte addressable)”이다.

즉, 각 주소는 1 바이트 정보를 가진다.

더블워드 변수는 32 비트(4 바이트)이므로 각 배열 요소는 4 바이트를 사용한다.

따라서:

- 두 번째 요소는 시작 주소 + 4
- 세 번째 요소는 시작 주소 + 8

각 요소마다 오프셋은 4 씩 증가한다.

오프셋 증가량은 데이터 크기에 따라 달라진다.:

- byte 배열: 1 증가
- word 배열: 2 증가
- double-word 배열: 4 증가
- quadword 배열: 8 증가

오프셋은 기준 주소에 더해지는 값이다.

인덱스는 고급 언어에서 사용하는 배열 요소 번호이다.

배열 요소에 접근하는 방법은 여러 가지가 있다.

기준 주소에 displacement 를 더하는 방법:

```
mov rbx, lst  
mov rsi, 8
```

다음 명령들은 모두 세 번째 요소(105)에 접근한다.

```
mov eax, dword [lst+8]  
mov eax, dword [rbx + 8]  
mov eax, dword [rbx+rsi]
```

각 경우 시작 주소 + 8 이 접근되며 105 가 eax 에 저장된다.

displacement 는 더해지지만 rbx 나 rsi 레지스터 값은 변경되지 않는다.

어떤 방법을 사용할지는 프로그래머 선택이다.

displacement 는 더 복잡한 방식으로 계산될 수 있다.

일반 메모리 주소 형식

```
[ baseAddr + (indexReg * scaleValue) + displacement ]
```

- baseAddr: 레지스터 또는 변수 이름
- indexReg: 반드시 레지스터
- scaleValue : 1, 2, 4, 8 중 하나의 즉시값
- 결과는 64 비트 주소

요소들은 조합하여 사용할 수 있으나 합법적이고 유용한 주소를 만들어야 한다.

예시

```
mov eax, dword [var1]
mov rax, qword [rbx+rsi]
mov aax, word [lst + 4]
mov bx, word [lst + rdx + 2]
mov rcx, word [lst + (rsi * 8)]
mov al, byte [buff-1+rcx]
mov eax, dword [rbx+(rsi*4)+16]
```

이전에 정의한 더블워드 배열의 세 번째 요소(index 2)를 접근하는 예:

```
mov rsi, 2
mov eax, dword [lst+rsi*4]
```

64 비트 아키텍처에서 주소는 항상 qword 이므로
더블워드 값을 접근하더라도
메모리 주소 계산에는 64 비트 레지스터가 사용된다.

이는 레지스터를 고급 언어의 배열 인덱스처럼 사용할 수 있게 한다.

예:

```
[lst+rsi*4]
```

는 고급 언어의

```
lst[rsi]
```

와 유사하다.

rsi 는 데이터 크기(이 예에서는 4)와 곱해진다.

[8-2] 예제 프로그램, 리스트 합계(Example Program, List Summation)

다음 예제 프로그램은 리스트에 있는 숫자들의 합을 구한다.

```
; Simple example to the sum and average for
; a list of numbers.
; *****
; Data declarations
```

```
section .data
; -----
; Define constants
EXIT_SUCCESS equ 0 ; successful operation
SYS_exit equ 60 ; call code for terminate
```



```
; -----
; Define Data.
section .data
lst dd 1002, 1004, 1006, 1008, 10010
len dd 5
sum dd 0
```

```
; *****
section .text
global _start
_start:
```

합계 루프

```
; -----
; Summation loop.
mov ecx, dword [len] ; get length value
mov rsi, 0           ; index=0
```

```
sumLoop:
mov eax, dword [lst+(rsi*4)] ; get lst[rsi]
add dword [sum], eax         ; update sum
inc rsi                      ; next item
loop sumLoop
```

프로그램 종료

```
; -----
; Done, terminate program.
last:
mov rax, SYS_exit          ; call code for exit
mov rdi, EXIT_SUCCESS      ; exit with success
syscall
```

대괄호 []안의 괄호 ()는 필수가 아니며, 명확성을 위해 추가된 것이다.
따라서 [lst+(rsi*4)]는 [lst+rsi*4]와 정확히 동일하다.

[8-2] 예제 프로그램, 피라미드의 면적과 부피

이 예제는 일련의 정사각뿔(square pyramid)에 대해 기하학적 정보를 계산하는 간단한 어셈블리 언어 프로그램이다.

구체적으로, 이 프로그램은 각 정사각뿔에 대해:

- 옆면을 포함한 전체 표면적(total surface area, 밑면 포함)
- 부피(volume)

를 계산한다.

값을 계산한 후, 프로그램은 전체 표면적과 부피에 대해:

- 최소값(minimum)
- 최대값(maximum)
- 합(sum)
- 평균(average)

을 구한다.

모든 데이터는 부호 없는 값(unsigned values)이다.
(즉, imul 이나 idiv 가 아니라, mul 과 div 를 사용한다.)

이 예제의 기본 접근 방식은 다음과 같다.

1. 첫 번째 루프에서 각 피라미드의 표면적과 부피 배열을 계산한다.
2. 두 번째 루프에서 각 배열의 합, 최소값, 최대값을 구한다.

최소값과 최대값을 구하는 방법은 다음과 같다.

- 최소값과 최대값 변수를 각각 리스트의 첫 번째 값으로 초기화한다.
- 리스트의 각 요소를 현재 최소값 및 최대값과 비교한다.
- 현재 값이 최소값보다 작으면 최소값을 현재값으로 갱신한다.
- 모든 값을 검사하면 최소값은 실제 최소값이 된다.
- 현재 값이 최대값보다 크면 최대값을 현재 값으로 갱신한다.
- 모든 값을 검사하면 최대값은 실제 최대값이 된다.

```
; Example assembly language program to calculate the  
; geometric information for each square pyramid in  
; a series of square pyramids.  
; The program calculates the total surface area  
; and volume of each square pyramid.  
; Once the values are computed, the program finds  
; the minimum, maximum, sum, and average for the  
; total surface areas and volumes.  
; -----
```

```
; Formulas:
; totalSurfaceAreas(n) = aSides(n) *
; (2*aSides(n)*sSides(n))
```

```
; volumes(n) = (aSides(n)^2 * heights(n)) / 3
; *****
section .data
```

상수 정의

```
; -----
; Define constants
EXIT_SUCCESS equ 0 ; successful operation
SYS_exit equ 60 ; call code for terminate
```

제공된 데이터

```
; -----
; Provided Data
aSides db 10, 14, 13, 37, 54
db 31, 13, 20, 61, 36
db 14, 53, 44, 19, 42
db 27, 41, 53, 62, 10
db 19, 18, 14, 10, 15
db 15, 11, 22, 33, 70
db 15, 23, 15, 63, 26
db 24, 33, 10, 61, 15
db 14, 34, 13, 71, 81
db 38, 13, 29, 17, 93

sSides dw 1233, 1114, 1773, 1131, 1675
dw 1164, 1973, 1974, 1123, 1156
dw 1344, 1752, 1973, 1142, 1456
dw 1165, 1754, 1273, 1175, 1546
dw 1153, 1673, 1453, 1567, 1535
dw 1144, 1579, 1764, 1567, 1334
dw 1456, 1563, 1564, 1753, 1165
dw 1646, 1862, 1457, 1167, 1534
dw 1867, 1864, 1757, 1755, 1453
dw 1863, 1673, 1275, 1756, 1353
```

```
heights dd 14145, 11134, 15123, 15123, 14123
dd 18454, 15454, 12156, 12164, 12542
dd 18453, 18453, 11184, 15142, 12354
dd 14564, 14134, 12156, 12344, 13142
dd 11153, 18543, 17156, 12352, 15434
```

```
dd 18455, 14134, 12123, 15324, 13453
dd 11134, 14134, 15156, 15234, 17142
dd 19567, 14134, 12134, 17546, 16123
dd 11134, 14134, 14576, 15457, 17142
dd 13153, 11153, 12184, 14142, 17134
```

```
length dd 50
taMin dd 0
taMax dd 0
taSum dd 0
taAve dd 0
volMin dd 0
volMax dd 0
volSum dd 0
volAve dd 0
```

추가 변수

```
ddTwo dd 2
ddThree dd 3
```

초기화되지 않은 데이터

```
section .bss
totalAreas resd 50
volumes resd 50
```

계산 루프

```
section .text
global _start
_start:
; Calculate volume, lateral and total surface areas
mov ecx, dword [length] ; length counter
mov rsi, 0 ; index

calculationLoop:
; totalAreas(n) = aSides(n) * (2*aSides(n)*sSides(n))
```

```

movzx r8d, byte [aSides+rsi]
movzx r9d, word [sSides+rsi*2]
mov eax, r8d
mul dword [ddTwo]
mul r9d
mul r8d
mov dword [totalAreas+rsi*4], eax

; volumes(n) = (aSides(n)^2 * heights(n)) / 3
movzx eax, byte [aSides+rsi]
mul eax
mul dword [heights+rsi*4]
div dword [ddThree]
mov dword [volumes+rsi*4], eax
inc rsi
loop calculationLoop

```

최소, 최대, 합, 평균 계산

```

mov eax, dword [totalAreas]
mov dword [taMin], eax
mov dword [taMax], eax

mov eax, dword [volumes]
mov dword [volMin], eax
mov dword [volMax], eax

mov dword [taSum], 0
mov dword [volSum], 0
mov ecx, dword [length]
mov rsi, 0

statsLoop:
mov eax, dword [totalAreas+rsi*4]
add dword [taSum], eax
cmp eax, dword [taMin]
jae notNewTaMin
mov dword [taMin], eax
notNewTaMin:
cmp eax, dword [taMax]
jbe notNewTaMax
mov dword [taMax], eax

```

```
notNewTaMax:
```

```
mov eax, dword [volumes+rsi*4]
add dword [volSum], eax
cmp eax, dword [volMin]
jae notNewVolMin
mov dword [volMin], eax
notNewVolMin:
cmp eax, dword [volMax]
jbe notNewVolMax
mov dword [volMax], eax
notNewVolMax:
inc rsi
loop statsLoop
```

평균 계산

```
mov eax, dword [taSum]
mov edx, 0
div dword [length]
mov dword [taAve], eax

mov eax, dword [volSum]
mov edx, 0
div dword [length]
mov dword [volAve], eax
```

프로그램 종료

```
last:
mov rax, SYS_exit
mov rdi, EXIT_SUCCESS
syscall
```

Chapter9. 프로세스 스택(Process Stack)

컴퓨터에서 스택(stack)은 항목을 추가한 후 역순으로 제거하는 데이터 구조의 한 형태이다.

즉, 가장 최근에 추가된 항목이 가장 먼저 제거된다.

이를 흔히 후입선출(Last-in, First-Out, LIFO)이라고 한다.

스택은 프로시저나 함수 호출 동안 정보를 저장하기 위해 프로그래밍에서 매우 많이 사용된다.

다음 장에서는 스택에 대한 정보와 예제를 제공한다.

스택에 항목을 추가하는 것을 push 또는 push 연산이라 한다.

스택에서 항목을 제거하는 것을 pop 또는 pop 연산이라 한다.

독자는 스택의 일반적인 개념에 익숙하다고 가정한다.

[9-1] Stack Example

스택의 일반적인 사용을 설명하기 위해 배열 $a = \{7, 19, 37\}$ 이 주어졌다고 하자.

다음 연산을 고려하자.

```
push a[0]
push a[1]
push a[2]
```

그 다음

```
pop a[0]
pop a[1]
pop a[2]
```

처음 push 는 7 을 스택에 넣고, 그 다음 19, 마지막으로 37 을 넣는다.

스택은 후입선출 구조이므로, 가장 먼저 pop 되는 값은 마지막에 push 된 값인 37 이다.

이 예에서 37 은 배열의 첫 번째 요소에 저장된다.(기존의 7 은 덮어쓴다.)

이 과정을 계속하면 배열 요소의 순서는 반대로 바뀐다.

다음 그림은 과정과 결과를 보여준다.

stack	stack	stack	stack	stack	stack
37					
19	19	19			
7	7	7	7	7	empty


```
push a[0]
push a[1]
push a[2]
pop a[0]
pop a[1]
```



```
a = {7, 19, 37}
a = {7, 19, 37}
a = {7, 19, 37}
a = {37, 19, 37}
a = {37, 19, 37}
```



```
pop a[2]
```



```
a = {37, 19, 7}
```

다음 절에서는 스택 구현과 관련된 스택 연산 및 명령어에 대해 더 자세히 설명한다.

[9-2] 스택 명령어(Stack Instructions)

push 연산은 값을 스택에 넣고,
pop 연산은 값을 스택에서 꺼낸다.

명령 형식은 다음과 같다.

```
push <operand64>
pop <operand64>
```

피연산자는 레지스터 또는 메모리일 수 있으나, 즉시값(immediate)은 허용되지 않는다.

일반적으로 push 와 pop 연산은 아키텍처 크기만큼 데이터를 처리한다.
아키텍처가 64 비트이므로 quadword(64 비트)를 push 하고 pop 한다.

스택은 메모리에서 역방향으로 구현된다.
그 이유는 다음 절에서 자세히 설명한다.

[9-2-1] push <op64>

64 비트 피연산자를 스택에 넣는다.

1. 먼저 rsp 를 조정한다($\text{rsp} - 8$)
2. 그 다음 피연산자를 [rsp]에 복사한다.

피연산자는 즉시값일 수 없다.

피연산자의 값은 변경되지 않는다.

예:

```
push rax
push qword [qVal]    ; 값
push qVal             ; 주소
```

[9-2-2] pop <op64>

스택에서 64 비트 값을 꺼낸다.

rsp 를 조정한다($\text{rsp} + 8$).

피연산자는 즉시값일 수 없다.

피연산자는 덮어써진다(overwritten).

예:

```
pop rax
pop qword [qVal]
pop rsi
```

64 비트보다 큰 값을 push 해야 하는 경우, 여러 번이 push 연산이 필요하다.

64 비트 보다 작은 피연산자를 push 하거나 pop 하는 것도 가능하지만 권장되지는 않는다.

더 완전한 명령어 목록은 부록에 있다.

[9-3] 스택 구현(Stack Implementation)

rsp 레지스터는 메모리에서 현재 스택의 최상단(top)을 가리키는 데 사용된다.
이 아키텍처에서는 대부분의 아키텍처와 마찬가지로,
스택은 메모리에서 아래 방향으로 성장하도록 구현되어 있다.

[9-3-1] 스택 구조(Stack Layout)

Chapter2, Architecture 에서 언급했듯이,
프로그램의 일반적인 메모리 구조는 다음과 같다.

high memory
stack
.
.
... available memory ...
.
.
heap
uninitialized data
data
text (code)
low memory
reserved

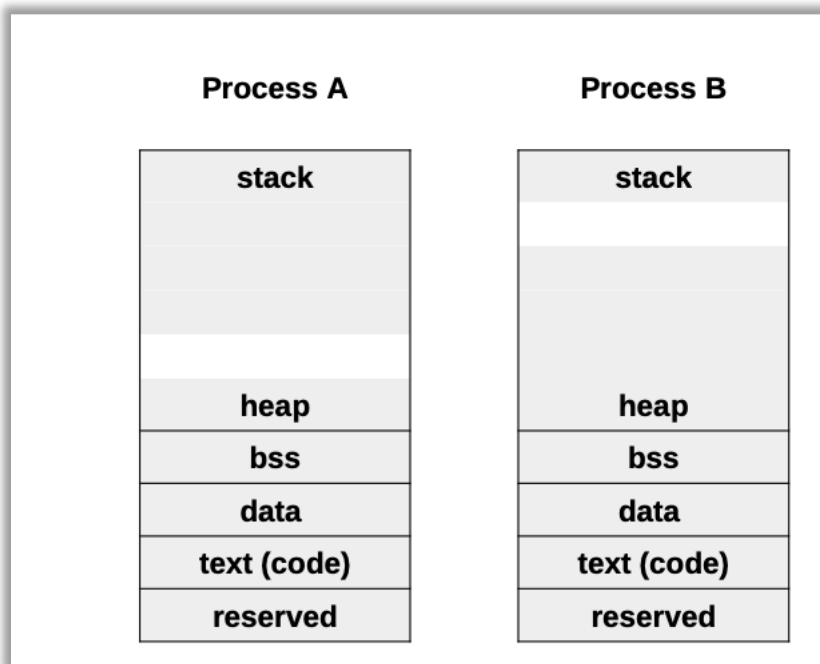
힙(heap)은 동적으로 할당된 데이터가 저장되는 영역이다(요청이 있을 경우).
예를 들어, C++의 new 연산자나 C의 malloc() 시스템 호출로 할당된 항목들이다.

동적으로 할당된 데이터는 실행 중(run-time)에 생성되므로, 힙은 일반적으로 위쪽 방향으로 성장한다.

반면, 스택은 높은 메모리 주소에서 시작하여 아래 방향으로 성장한다.
스택은 함수 호출의 호출 프레임(call frame)과 같은 임시 정보를 저장하는 데 사용된다.
큰 프로그램이나 재귀함수는 상당한 양의 스택 공간을 사용할 수 있다.

힙과 스택이 확장될 때, 서로를 향해 성장한다.
이는 메모리를 효율적으로 사용하기 위해서이다.

스택을 많이 사용하고 힙을 거의 사용하지 않는 프로그램(Process A)은 정상적으로 동작할 수 있다. 스택을 거의 사용하지 않고 힙으로 매우 많이 사용하는 프로그램(Process B)도 정상적으로 동작할 수 있다.



물론, 스택과 힙이 서로 만나게 되면 프로그램은 충돌(crash)한다.
그 경우 더 이상 사용 가능한 메모리가 없다.

[9-3-2] 스택 연산 (Stack Operations)

기본적인 스택 연산인 push와 pop은 연산 중에 스택 포인터 레지스터 rsp를 조정한다.

[1] push 연산

1. rsp 레지스터를 8 감소시킨다.(1 quad word)
2. 피연산자를 [rsp] 위치에 복사한다.

피연산자의 값은 변경되지 않는다. 이 연산들의 순서는 중요하다.

[2] pop 연산

1. 현재 스택의 최상단 [rsp]에 있는 값을 피연산자로 복사한다.
2. rsp 레지스터를 8 증가시킨다.(1 quad word)

이 연산의 순서는 push의 정확한 역순이다.

pop된 항목은 실제로 삭제되지 않는다.

그러나 pop 이후에도 그 값이 스택에 남아 있을 것이라고 프로그래머가 기대해서는 안된다.
이전에 push되었지만 아직 pop되지 않은 항목들은 접근할 수 있다.

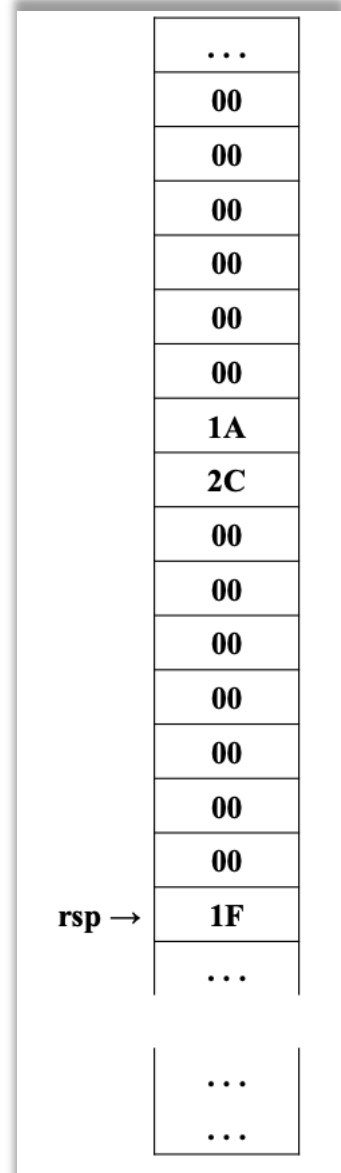
예:

```
mov rax, 6700    ; 670010 = 0x00001A2C
push rax
mov rax, 31      ; 3110 = 0x0000001F
push rax
```

다음과 같은 스택 구성을 생성한다.

이 구조는 아키텍처가 리틀 엔디안임을 보여준다.

즉, 최하위 바이트가 가장 낮은 메모리 주소에 저장된다.



[9-4] 스택 예제

다음은 스택을 사용하여 quadword 리스트를 제자리(in place)에서 뒤집는 예제 프로그램이다. 구체적으로, 첫 번째 루프에서 quadword 배열의 각 값을 스택에 넣는다. 두 번째 루프에서는 각 요소를 스택에 꺼내어 배열에 다시 저장하며 이대 이전 값을 덮어쓴다.

```
; Simple example demonstrating basic stack operations.
; Reverse a list of numbers - in place.
; Method: Put each number on stack, then pop each number
; back off, and then put back into memory.
; *****
; Data declarations
```

```
section .data
; -----
; Define constants
EXIT_SUCCESS equ 0 ; successful operation
SYS_exit equ 60 ; call code for terminate
; -----
; Define Data.
numbers dq 121, 122, 123, 124, 125
len dq 5
```

```
; *****

section .text
global _start
_start:
; 숫자를 스택에 넣는 루프
; Loop to put numbers on stack.
mov rcx, qword [len]
mov rbx, numbers
mov r12, 0
mov rax, 0

pushLoop:
push qword [rbx+r12*8]
inc r12
loop pushLoop
```

```
; 모든 숫자가 스택에 있음(역순 상태)
; -----
; All the numbers are on stack (in reverse order).
; Loop to get them back off. Put them back into
; the original list...
mov rcx, qword [len]
mov rbx, numbers
mov r12, 0

popLoop:
pop rax
mov qword [rbx+r12*8], rax
inc r12
loop popLoop

;프로그램 종료
; -----
; Done, terminate program.
last:
mov rax, SYS_exit          ; call code for exit
mov rdi, EXIT_SUCCESS      ; exit with success
syscall
```

Chapter10. 프로그램 개발(Program Development)

명확한 방법론을 따를 때 프로그램 작성 또는 개발은 더 쉬워진다.

이 방법론의 주요 단계는 다음과 같다.

- 문제 이해
- 알고리즘 작성
- 프로그램 구현
- 프로그램 테스트/디버깅

이 과정을 자세히 설명하기 위해, 다음 절에서는 간단한 예제 문제에 이 단계들을 적용한다.

[10-1] 문제 이해

해결책을 만들기 전에, 문제를 완전히 이해하는 것이 중요하다.

문제를 완전히 이해하면 오류를 줄이고, 시간과 노력을 절약할 수 있다.

첫 번째 단계는 무엇이 요구되는지를 이해하는 것이다.

특히 입력 정보와 기대되는 결과 또는 출력이 무엇인지 이해해야 한다.

하나의 정수(integer)를 그 정수를 나타내는 문자열 또는 일련의 문자로 변환하는 문제를 생각하자.

명확히 하면:

- 정수(integer)는 수치 계산에 사용할 수 있지만 그대로는 콘솔에 출력할 수 없다.
- 문자열(string)은 콘솔에 출력할 수 있지만 수치 계산에는 사용할 수 없다.

이 예제에서는 부호 없는 값(양수만)을 고려한다.

부호 있는 값을 처리하는 약간의 추가 작업은 독자의 연습 문제로 남겨 둔다.

부호 없는 double-word 정수로서, 숫자 값은 1498 은 16 진수로 0x000005DA(double-word 크기)로 표현된다.

정수 1498(0x000005DA)는 문자열 “1”, “4”, “9”, “8” 그리고 NULL 문자로 표현된다.

이 경우 부호나 앞의 공백이 필요 없으므로 총 5 바이트가 필요하다.

따라서, 문자열 “1498”은 다음과 같이 표현된다.

Character	“1”	“4”	“9”	“8”	NULL
ASCII Value (decimal)	49	52	57	56	0
ASCII Value (hex)	0x31	0x34	0x39	0x38	0x0

목표는 하나의 정수를 적절한 일련의 문자로 변환하여 NULL 종료 문자열(NULL terminated string)을 만드는 것이다.

[10-2] 알고리즘 작성

알고리즘이란 문제를 해결하기 위해 필요한 명확하고, 순서가 있는 단계들의 집합을 말한다.
문제를 이해한 후에는 그 문제를 해결하기 위한 일련의 단계를 개발할 수 있다.

하나의 문제에 대해 여러 개의 올바른 해결 방법이 존재할 수 있으며,
보통 실제로도 여러 가지가 존재한다.
알고리즘을 만드는 과정은 사람마다 다를 수 있다.

일반적으로 가능한 해결책에 대해 생각하는 데 시간을 투자해야 한다.
이 과정은 종이에 여러 가지 가능한 해결 방법을 시도해 보는 것을 포함할 수 있다.

하나의 접근 방식을 선택한 후, 그 해결 방법을 알고리즘으로 발전시킬 수 있다.
알고리즘은 작성되고, 검토되며, 다듬어져야 한다.
그리고 그 알고리즘은 프로그램의 개요로 사용된다.

이전 절에서 설명한 정수를 ASCII 로 변환하는 문제를 예로 들어보자.
한 자리 정수(0~9)를 문자로 변환하려면
48(또는 "0" 또는 0x30)을 그 정수에 더하면 된다.

예를 들어,

$0x01 + 0x30 = 0x31$

이는 문자 "1"의 ASCII 값이다.

이 방법은 한 자리 숫자(0~9)에만 적용된다는 것은 명확하다.

더 큰 정수(≥ 10)를 문자열로 변환하려면 정수를 각각의 자릿수로 분해해야 한다.

예를 들어,

$123(0x7B) \rightarrow 1, 2, 3$

이는 10 으로 반복적으로 나누는 정수 나눗셈을 수행하여 몫이 0 이 될 때까지 반복하면 된다.

예:

$123 / 10 = 12 \text{ remainder } 3$
$12 / 10 = 1 \text{ remainder } 2$
$1 / 10 = 0 \text{ remainder } 1$

여기서 볼 수 있듯이, 나머지(remainder)가 각각의 자릿수를 나타낸다.

그러나 이 값들은 역순으로 얻어진다.

이를 해결하기 위해 프로그램은 나머지를 push 하고 나눗셈이 끝난 후 pop 하여
ASCII 로 변환한 뒤 문자열(바이트 배열)에 저장할 수 있다.

이 과정이 알고리즘의 기초를 형성한다.

이 알고리즘을 개발하는 방법은 여러 가지가 있다.

그 중 하나는 다음과 같다.

```
; Part A – Successive division
; digitCount = 0
; get integer
; divideLoop:
;     divide number by 10
;     push remainder
;     increment digitCount
;     if (result > 0) goto divideLoop
```

```
; Part B – Convert remainders and store
; get starting address of string (array of bytes)
; idx = 0
; popLoop:
;     pop intDigit
;     charDigit = intDigit + "0" (0x030)
;     string[idx] = charDigit
;     increment idx
;     decrement digitCount
;     if (digitCount > 0) goto popLoop
; string[idx] = NULL
```

이 알고리즘 단계들은 편의를 위해 프로그램 주석 형태로 제시되었다.

일반적으로 알고리즘은 종이에 먼저 작성하고,

그 후 위와 같이 보다 형식적인 의사코드(pseudo-code)로 작성된다.

프로그램이 처음에 제대로 동작하지 않는 경우,

이 주석들이 주요 디버깅 체크리스트가 된다.

일부 프로그래머는 주석을 생략하지만, 그 결과 디버깅에 훨씬 더 많은 시간을 소비하게 된다.

주석은 알고리즘을 나타내며, 코드는 그 알고리즘의 구현이다.

[10-3] 프로그램 구현

알고리즘을 기반으로 프로그램을 개발하고 구현할 수 있다.

알고리즘에서 제시된 단계들을 확장하여 코드를 추가한다.

이렇게 하면 프로그래머는 현재 작성 중인 부분의 구체적인 문제, 예를 들어, 데이터 타입과 데이터 크기에 집중할 수 있다.

이 예제는 부호 없는 데이터만을 다루므로 부호 없는 나눗셈(div)을 사용한다.

정수가 double-word 이므로 나눗셈을 위해 quadword 로 변환되어야 한다.

그러나 나눗셈 이후의 몫과 나머지는 또한 double-word 가 된다.

스택은 quadword 단위이므로 전체 quadword 레지스터가 push 된다.

레지스터의 상위 부분은 접근되지 않으므로 그 내용은 중요하지 않다.

이 알고리즘의 한 가지 구현 예는 다음과 같다.

```
; Simple example program to convert an
; integer into an ASCII string.
; *****
; Data declarations
section .data
; -----
; Define constants
NULL equ 0
EXIT_SUCCESS equ 0 ; successful operation
SYS_exit equ 60     ; code for terminate
; -----
; Define Data.
intNum dd 1498

section .bss
strNum resb 10

; *****
section .text
global _start
```

정수를 ASCII 문자열로 변환

Part A - 연속 나눗셈

Part B - 나머지 변환 및 저장

```
_start:
mov eax, dword [intNum] ; get integer
mov rcx, 0               ; digitCount = 0
mov ebx, 10              ; set for dividing by 10

divideLoop:
mov edx, 0
div ebx                  ; divide number by 10
push rdx                 ; push remainder
inc rcx                  ; increment digitCount
cmp eax, 0               ; if (result > 0)
jne divideLoop           ; goto divideLoop

mov rbx, strNum           ; get addr of string
mov rdi, 0                ; idx = 0

popLoop:
pop rax                  ; pop intDigit
add al, "0"              ; char = int + "0"
mov byte [rbx+rdi], al   ; string[idx] = char
inc rdi                  ; increment idx
loop popLoop             ; if (digitCount > 0)

; goto popLoop
mov byte [rbx+rdi], NULL ; string[idx] = NULL
```

프로그램 종료

```
; -----
; Done, terminate program.
last:
mov rax, SYS_exit        ; call code for exit
mov rdi, EXIT_SUCCESS    ; exit with success
syscall
```

이 알고리즘에는 여러 가지 다른 올바른 구현 방법이 존재한다.
프로그램은 오타나 문법 오류를 확인하기 위해 어셈블되어야 한다.

[10-4] 프로그램 테스트/디버깅

프로그램이 작성되면, 프로그램이 올바르게 동작하는지 확인하기 위해 테스트를 수행해야 한다.

테스트는 프로그램의 구체적인 요구 사항에 따라 수행된다.

이 경우, 디버거를 사용하여 프로그램을 실행하고 프로그램의 끝 부분(예: 이 예제에서는 “last” 라벨 위치) 근처에서 중단할 수 있다.

ddd 로 디버거를 시작한 후, 다음 명령을 입력할 수 있다.

```
b last  
run
```

이 명령은 프로그램을 실행하여 “last” 라벨이 가리키는 줄을 실행하기 직전까지 실행한다.

결과 문자열 strNum 은 다음 디버거 명령으로 확인할 수 있다.

```
x/s &strNum
```

이 명령은 문자열의 주소와 그 내용을 표시하며 값은 “14998”이어야 한다.

예:

```
(gdb) x/s &strNum  
0x600104: “1498”
```

문자열이 올바르게 표시되지 않는 경우, 디버거 명령 x/5cb &strNum 을 사용하여 5 바이트 배열의 각 문자를 확인해 볼 수 있다.

출력은 문자열의 주소와 함께 각 바이트의 10 진수 값과 ASCII 표현을 보여준다.

예:

```
(gdb) x/5cb &strNum  
0x600104: 49 '1' 52 '4' 57 '9' 56 '8' 0 'W000'
```

이 출력 형식은 처음에는 혼란스러울 수 있다.

올바른 출력이 나오지 않는 경우, 프로그래머는 코드를 디버깅해야 한다.

이 예제에는 두 가지 주요 단계가 있다.

1. 연속 나눗셈
2. 나머지 변환 및 저장

두 번째 단계는 첫 번째 단계가 올바르게 동작해야 가능하므로, 먼저 첫 번째 단계를 검증해야 한다. 이는 디버거를 사용하여 첫 번째 부분만 집중적으로 확인함으로써 수행할 수 있다.

이 예제에서 첫 번째 단계는 정확히 4 번 반복되어야 하므로 rcx 값은 4 여야 한다.
또한, 8, 9, 4, 1 이 그 순서대로 스택에 push 되어야 한다.

이는 rdx 레지스터의 내용을 push 시점에 확인하거나
스택의 상위 4 개 항목을 확인함으로써 쉽게 검증할 수 있다.

그 부분이 올바르게 동작한다면, 두 번째 부분을 검증할 수 있다.
이 단계에서는 1, 4, 9, 8 이 그 순서대로 스택에서 pop 되어야 한다.

그렇다면 정수는 “0”(0x30)을 더함으로써 문자로 변환되고, 그 문자는 한 글자씩 문자열에
저장된다.
문자열을 문자 단위로 확인하여 각 문자가 올바르게 저장되고 있는지 확인할 수 있다.

이와 같은 방식으로 문제의 원인을 비교적 빠르게 좁혀 나갈 수 있다.
효율적인 디버깅은 매우 중요한 기술이며, 연습을 통해 숙달되어야 한다.
구체적인 디버거 명령에 대한 추가 정보는 스스로 찾아보자.

[10-5] 오류 용어

프로그램이 제대로 동작하지 않는 경우, 오류가 어디에 있는지 또는 어떤 종류인지에 대한
기본 용어를 이해하는 것이 도움이 된다.
올바른 용어를 사용하는 것은 문제에 대해 다른 사람과 효과적으로 의사소통하는 데
중요하다.

[10-5-1] Assembler Error(어셈블러 오류)

어셈블러 오류는 프로그램을 어셈블할 때 발생한다.
이는 어셈블러가 하나 이상의 명령을 이해하지 못한다는 의미이다.
어셈블러는 오류 목록과 각 오류의 줄 번호를 제공한다.

오류는 위에서 아래로(top down) 순서대로 수정하는 것이 권장된다.
위쪽의 오류를 해결하면 아래쪽에 나타난 여러 오류가 함께 해석될 수 있다.

일반적인 어셈블러 오류에는
명령어의 철자 오류나 변수 선언 누락이 포함된다.

[10-5-2] Run-time Error(실행시간 오류)

실행 시간 오류는 프로그램이 실행 중에 충돌(crash)하게 만드는 오류이다.

[10-5-3] Logic Error(논리 오류)

논리 오류는 프로그램이 실행되지만
올바른 결과를 생성하지 않는 경우를 말한다.

예를 들어:

- 주어진 공간을 잘못 코딩한 경우
- 숫자들의 합을 계산하기 전에 평균을 계산하려는 경우

프로그램에 논리 오류가 있는 경우, 오류를 찾는 한 가지 방법은 중간 값을 출력해보는 것이다.

논리 오류를 찾는 방법에 대한 추가 정보는 이후 장에서 제공될 것이다.

Chapter11. 매크로(Macros)

어셈블리 언어 매크로는 필요한 곳 어디에나 쉽게 삽입할 수 있는 미리 정의된 명령어 집합이다.

한 번 정의되면, 그 매크로는 필요한 만큼 여러 번 사용할 수 있다.

같은 코드 집합을 여러 번 사용해야 할 때 매우 유용하다.

매크로는 코드 양을 줄이고,
프로글매를 단순화하며,
반복 코딩으로 인한 오류를 줄이는 데 도움이 된다.

어셈블러는 강력한 매크로 프로세서를 포함하고 있으며, 다음 기능들을 지원한다:

- 조건부 어셈블리
- 다단계 파일 포함
- 두 가지 형태의 매크로(단일 행, 다중 행)
- 추가적인 매크로 기능을 위한 '컨텍스트 스택(context stack)'메커니즘

매크로를 사용하기 전에 반드시 정의해야 한다.

매크로 정의는 데이터 섹션과 코드 섹션보다
앞에 소스 파일 상단에 위치해야 한다.

매크로 텍스트(코드) 섹션에서 사용된다.

다음 절에서는 정의와 사용을 포함한 자세한 예제를 제시한다.

[11-1] 단일 행 매크로(Single-Line Macros)

매크로에는 두 가지 주요 유형이 있다.

- 단일 행 매크로(single-line macros)
- 다중 행 매크로(multi-line macros)

각각은 다음 절에서 설명된다.

단일 행 매크로는 `%define` 지시어를 사용하여 정의된다.

이 정의 방식은 C/C++와 유사하게 동작한다.

예를 들어, 다음과 같이 작성할 수 있다.

```
#define mulby4(x) shl x, 2
```

그리고 소스 코드에서 다음과 같이 매크로를 사용할 수 있다.

```
mulby4 (rax)
```

이 코드는 rax 레지스터의 내용을 4 배로 만든다.
(두 비트를 왼쪽으로 시프트함으로써)

[11-2] 다중 행 매크로(Multi-Line Macros)

다중 행 매크로는 여러 줄(한 줄 포함)의 명령을 포함할 수 있다.

다중 행 매크로는 더 유용하며, 다음절에서는 주로 다중 행 매크로에 초점을 맞춘다.

[11-2-1] 매크로 정의(Macro Definition)

다중 행 매크로를 사용하기 전에
먼저 정의해야 한다.

일반 형식은 다음과 같다.

```
%macro <name> <number of arguments>  
; [body of macro]  
%endmacro
```

매크로 내부에서 인수(arguments)는
%<번호> 형태로 참조할 수 있다.

- %1: 첫 번째 인수
- %2: 두 번째 인수
- 그 이후도 같은 방식

라벨을 사용하려면

매크로 내부의 라벨 이름 앞에 %%를 붙여야 한다.

이렇게 하면 동일한 매크로를 여러 번 호출하더라도 각 호출마다 서로 다른 라벨이 사용된다.
예를 들어, 절댓값 함수 매크로 정의는 다음과 같다.

```
%macro abs 1  
cmp %1, 0  
jge %%done  
neg %1  
%%done:  
%endmacro
```

전체 예제는 샘플 매크로 프로그램을 참고하자.

[11-2-2] 매크로 사용(Using a Macro)

매크로를 사용(또는 “호출”)하려면

코드 영역에 배치하고 적절한 개수의 인수와 함께 이름으로 호출해야 한다.

다음과 같은 데이터 선언이 있다고 가정하자.

```
qVar d dq 4
```

그렇다면 “abs” 매크로를 두 번 호출하려면:

```
mov eax, -3
```

```
abs eax
```

```
abs qword [qVar]
```

리스트 파일에는 (첫 번째 호출에 대해) 다음과 같이 표시된다.

```
27 00000000 B8FDFFFFFF mov eax, -3
```

```
28 abs eax
```

```
29 00000005 3D00000000 <1> cmp %1, 0
```

```
30 0000000A 7D02 <1> jge %%done
```

```
31 0000000C F7D8 <1> neg %1
```

```
32 <1> %%done:
```

매크로는 정의된 내용을 코드로 복사하며, 사용될 때마다 매크로 본문에서 인수가 실제 값으로 대체된다.

<1> 표시는 매크로 정의에서 복사된 코드임을 의미한다.

이 예제에서는 두 경우 모두

%1 인수가 eax 로 대체되었다.

매크로는 더 많은 메모리를 사용하지만,

함수와 달리 제어 이동(transfer of control)에 따른 오버헤드는 필요하지 않다.

[11-3] 매크로 예제(Macro Example)

다음 예제 프로그램은 간단한 매크로의 정의와 사용을 보여준다.

```
; Example Program to demonstrate a simple macro
; *****
; Define the macro
; called with three arguments:
; aver <lst>, <len>, <ave>
```

```
%macro aver 3
mov eax, 0
mov ecx, dword [%2] ; length
mov r12, 0
lea rbx, [%1]
%%sumLoop:
add eax, dword [rbx+r12*4] ; get list[n]
inc r12
loop %%sumLoop
cdq
idiv dword [%2]
mov dword [%3], eax
%endmacro
```

```
데이터 선언
; *****
; Data declarations
section .data
; -----
; Define constants
EXIT_SUCCESS equ 0 ; success code
SYS_exit equ 60 ; code for terminate

; Define Data.
section .data
list1 dd 4, 5, 2, -3, 1
len1 dd 5
ave1 dd 0

list2 dd 2, 6, 3, -2, 1, 8, 19
len2 dd 7
ave2 dd 0
```

```
; *****  
section .text  
global _start
```

```
_start:  
aver list1, len1, ave1 ; 1st, data set 1  
aver list2, len2, ave2 ; 2nd, data set 2  
  
; -----  
; Done, terminate program.  
last:  
mov rax, SYS_exit      ; exit  
mov rdi, EXIT_SUCCESS  ; success  
syscall
```

이 예제에서 매크로는 두 번 호출된다.
매크로가 사용될 때마다,
정의된 내용이 텍스트(section .text)영역으로 복사된다.

따라서 매크로는 일반적으로 더 많은 메모리를 사용한다.

[11-4] 매크로 디버깅(Debugging Macros)

매크로의 코드는 디버깅 소스 창에 표시되지 않는다.

매크로가 정상적으로 동작할 때는 이것이 매우 편리하다.
그러나 매크로를 디버깅할 때는 코드가 보일 필요가 있다.

매크로 코드를 확인하려면 Machine Code Window 를 표시해야 한다.
(View → Machine Code Window)

이 창에서는 각 명령의 머신 코드가 표시된다.
step 및 next 명령은 매크로 전체를 한 번에 실행한다.

매크로 내부의 각 명령을 하나씩 실행하려면 stepi 및 nexti 명령을 사용해야 한다.
표시되는 코드는 원래의 매크로 정의가 아니라 확장된 코드 형태로 나타난다.

Chapter12. 함수(Functions)

함수(functions)와 프로시저(void 함수)는 프로그램을 논리적인 단위로 나누기 위한 핵심 도구이다. 하나의 거대한 코드 덩어리로 프로그램을 작성하면 이해하기 어렵고, 오류를 찾기도 힘들며, 수정 또한 복잡해진다.

함수를 사용하면:

- 기능 단위로 코드를 분리할 수 있고
- 반복되는 코드를 재사용할 수 있으며
- 디버깅과 유지보수가 쉬워진다.

함수 호출이 제대로 동작하기 위해서는 두 가지 중요한 동작이 필요하다.

링크(Linkage)

함수는 프로그램의 여러 위치에서 호출될 수 있다.

따라서 함수는 실행이 끝난 뒤 자신을 호출한 정확한 위치로 되돌아가야 한다.

이를 위해 컴퓨터는 “반환 주소(return address)”를 저장한다.

링키지는 바로 이 과정을 처리하는 메커니즘이다

인수 전달(Argument Transmission)

함수는 단순히 실행되는 코드 블록이 아니라, 입력을 받아 처리하고 결과를 반환하는 연산 단위이다.

따라서 함수는:

- 매개변수(parameter)에 접근할 수 있어야 하고
- 필요한 연산을 수행해야 하며
- 경우에 따라 값을 호출자에게 되돌려 주어야 한다.

특히 call-by-reference 방식의 경우, 함수는 전달된 메모리 위치에 직접 접근하여 값을 변경할 수 있다.

이러한 두 동작(링키지와 인수 전달)이 어떻게 구현되는지 다음 절에서 설명한다.

[12-1] 업데이트된 링크 명령

어셈블리 프로그램을 작성할 때는 C 컴파일러(gcc 또는 g++)를 사용하여 링크하는 것이 일반적이다. 이는 C/C++ 표준 라이브러리의 위치와 링커 설정을 GCC가 자동으로 처리해주기 때문이다.

예를 들어, 소스 파일이 example.asm 이라면 다음과 같이 빌드한다.

1 단계: 어셈블

```
yasm -g dwarf2 -f elf64 example.asm -l example.lst
```

이 명령은:

- example.asm 을 example.o 오브젝트 파일로 변환한다.
- -g 옵션은 디버깅 정보를 포함시킨다.

2 단계: 링크

```
gcc -g -o example example.o
```

Ubuntu 18 이상에서는 다음과 같이 작성해야 한다.

```
gcc -g -no-pie -o example example.o
```

-no-pie 옵션은 위치 독립 실행 파일(PIE) 생성을 방지하기 위한 것이다.

최근 Linux 시스템에서는 기본적으로 PIE 가 활성화되어 있기 때문에 이 옵션이 필요하다. 결과적으로 GCC 는 내부적으로 링커를 호출하여 example 실행 파일을 생성한다.

[12-2] 디버깅 명령

함수를 포함하는 프로그램을 디버깅할 때는

디버거 명령 next 와 step 의 차이를 정확히 이해하는 것이 중요하다.

[12-2-1] 디버거 명령 next

next 명령은 함수 호출을 하나의 단위로 간주한다.

즉,

- 함수 내부로 들어가지 않고
- 함수 전체를 실행한 뒤
- 다음 줄로 이동한다.

이 명령은 함수 내부가 이미 검증되었을 때, 결과만 빠르게 확인하고 싶을 때 유용하다.

[12-2-1] 디버거 명령 step

step 명령은 함수 호출 시 내부로 진입한다.

즉,

- 함수의 첫 줄로 이동하고
- 한 줄씩 실행한다.

이 명령은 함수 내부 동작을 상세히 분석하거나 오류를 찾고자 할 때 매우 유용하다.

[12-3] 스택 동적 지역 변수

고급 언어(C/C++)에서 함수 내부에 선언된 비정적(non-static) 지역 변수는 기본적으로 스택 동적 지역 변수(stack-dynamic local variable)이다.

C++교재에서는 이를 automatic 변수라고 부르기도 한다.

동작 원리

함수가 호출되면:

1. 스택에 공간이 할당되고
2. 그 공간이 지역 변수에 지정된다.
3. 함수가 종료되면
4. 해당 스택 공간은 회수된다.

이 방식은 약간의 실행 시간 오버헤드를 요구하지만, 메모리를 매우 효율적으로 사용한다.

메모리 효율성

만약 어떤 함수가 한 번도 호출되지 않는다면,
그 함수의 지역 변수는 아예 메모리를 차지하지 않는다.
이는 프로그램 전체 메모리 사용량을 줄이고
성능 향상에도 기여한다.

정적 변수와의 비교

정적으로 선언된 변수(static 변수)는
프로그램이 메모리에 로드될 때 이미 공간이 할당된다.

- 프로그램 전체 실행 기간 동안 유지
- 함수가 호출되지 않아도 메모리 사용
- 실행 중 추가 할당 비용 없음

즉, 프로그램 시작 시 한 번에 메모리를 확보해 두기 때문에
실행 도중 메모리 할당으로 인한 지연이 발생하지 않는다.

[12-4] 함수 선언

함수는 사용되기 전에 먼저 작성되어야 한다.
어셈블리 에서는 함수는 코드 세그먼트에 위치한다.

기본 형식은 다음과 같다.

```
global <procName>
<procName>:
; function body
ret
```

규칙

- 함수는 한 번만 정의될 수 있다.
- 함수는 중첩될 수 없다.
- 하나의 함수 정의가 완전히 끝난 뒤에야 다음 함수를 정의할 수 있다.
- 정의 순서에는 특별한 제한이 없다.

[12-5] 표준 호출 규약

어셈블리 프로그램에서 함수 간 상호작용을 원활히 하기 위해 표준 호출 규약(Calling Convention)이 필요하다.

만약 각 함수가:

- 매개변수 전달 방식을 다르게 사용하고
- 서로 다른 레지스터를 임의로 변경한다면

프로그램은 매우 혼란스러워질 것이다.

이를 방지하기 위해 표준 절차가 정의되어 있다.

[12-5-1] System V AMD64 ABI

이 문서에서 사용하는 호출 규약은

64 비트 Linux 환경의 표준 규약인 System V AMD64 ABI 이다.

이 규약은 C/C++ 프로그램에서도 사용되므로,

어셈블리 코드와 C/C++ 코드 간의 인터페이스를 쉽게 구현할 수 있다.

단, Microsoft Windows 의 호출 규약은 다르며, 여기서는 다루지 않는다.

[12-6] 링키지(Linkage)

링키지는 함수 호출과 복귀를 정확히 처리하는 메커니즘이다.

이를 수행하는 명령은 두 가지이다.

```
call <funcname>
```

```
ret
```

[12-6-1] call 명령

call 명령은 다음과 같이 동작한다.

1. 현재 rip 레지스터의 값(다음 실행 명령 주소)을 스택에 저장(push)
2. 지정된 함수로 점프

rip 는 다음에 실행될 명령을 가리키는 레지스터이다

따라서 call 은 복귀 지점을 스택에 보관하는 역할을 한다.

[12-6-2] ret 명령

ret 명령은 다음과 같이 동작한다.

1. 스택의 최상단 값을 pop
2. 해당 값을 rip 에 저장
3. 원래 호출 지점으로 점프

즉, call 이 짚아한 반환 주소를 ret 이 복원하는 구조이다.

[12-6-3] 스택 손상에 대한 주의

링키지는 스택을 기반으로 동작한다.

따라서 함수 내에서 스택 균형이 깨지면 안된다.

예를 들어,

- 값을 push 만 하고 pop 하지 않으면
- ret 명령은 잘못된 값을 rip 에 로드하게 된다.

그 결과:

- 잘못된 코드 실행
- 프로그램 충돌
- segmentation fault 발생

따라서 함수 내부에서는 push 한 값을 반드시 pop 해야 한다.

[12-6-4] 함수 호출 요약

Instruction	Explanation
call funcName	rip를 스택에 push하고 funcName으로 점프
ret	스택 값을 rip에 pop하여 호출 위치로 복귀

[12-7] 인수 전달(Argument Transmission)

인수 전달이란

함수를 호출할 때 필요한 정보(변수, 값, 주소 등)를 함수에 전달하고, 필요한 경우 결과를 되돌려 받는 과정을 의미한다.

함수는 독립된 코드 블록이기 때문에,

외부에서 아무 정보도 받지 못한다면 의미 있는 연산을 수행할 수 없다.

따라서 인수 전달은 함수 호출 메커니즘의 핵심 요소 중 하나이다.

[12-7-1] call-by-value

값을 함수에 전달하는 방식을 call-by-value 라고 한다.

이 방식에서는

- 원래 변수의 값이 복사되어
- 함수의 매개변수로 전달된다.

즉, 함수 내부에서 매개변수를 수정하더라도 원래 변수에는 아무런 영향이 없다.

예(개념적 설명):

```
int x = 10;  
func(x);
```

이 경우 함수는 10 이라는 값의 복사본을 받는다.

함수 내부에서 값을 20 으로 변경해도 x 는 여전히 10 이다.

[12-7-2] call-by-reference

주소를 함수에 전달하는 방식을 call-by-reference 라고 한다.

이 방식에서는:

- 변수의 실제 메모리 주소가 전달된다.
- 함수는 그 주소를 통해 원래 변수에 직접 접근한다.

따라서 함수 내부에서 값을 변경하면

호출한 쪽의 변수도 함께 변경된다.

예(개념적 설명):

```
int x = 10;  
func(&x);
```

함수는 x 의 주소를 받아 해당 메모리 위치의 값을 직접 수정할 수 있다.

이 두 개념은 고급 언어(C, C++, Java 등)에서 이미 익숙한 내용일 것이다.

어셈블리 언어에서는 이 개념이 레지스터, 스택, 메모리 주소 수준에서 직접 구현된다.

[12-7-3] 인수 전달 방법

함수에 인수를 전달하거나 함수로부터 결과를 받는 방법은 여러 가지가 있다.

각 방법은 장단점을 가지며, 시스템의 호출 규약에 따라 표준화되어 있다.

[1] 레지스터에 값 저장

가장 빠르고 효율적인 방법은 레지스터를 이용하여 값을 전달하는 것이다

특징

- 접근 속도가 가장 빠름
- 구현이 단순함
- 하지만 레지스터 개수에 제한이 있음

64 비트 Linux 의 System V AMD64 ABI 에서는:

- 처음 여섯 개의 정수형 인수는 레지스터를 통해 전달된다.
- 시스템 호출(syscall)에서도 동일한 방식이 사용된다.

장점

- 메모리 접근이 필요 없음
- 실행 속도 빠름

단점

- 레지스터 수만큼만 직접 전달 가능
- 많은 인수를 전달할 경우 추가 방법 필요

[2] 전역 변수 사용

전역 변수를 통해 값을 공유하는 방법도 존재한다.

특징

- 함수 외부에 선언된 변수를 직접 사용
- 인수 전달 과정이 필요 없음

그러나 이 방법은 일반적으로 권장되지 않는다.

이유

- 코드 의존성이 강해짐
- 여러 함수가 동일한 전역 변수를 수정할 경우 혼란 발생
- 재사용성과 모듈성이 크게 떨어짐
- 다중 스레드 환경에서 문제 발생 가능

따라서 전역 변수는 제한된 상황에서만 사용해야 한다.

[3] 값 또는 주소를 스택에 저장

레지스터만으로는 인수 개수가 부족할 수 있다.
이 경우 스택을 이용하여 인수를 전달한다.

특징

- 전달 가능한 인수 개수에 특별한 제한이 없음
- 레지스터를 초과하는 인수는 스택에 저장됨

동작 개념

1. caller 가 값을 스택에 push
2. callee 가 스택에서 해당 값을 읽음

장점

- 많은 수의 인수 전달 가능
- 구조체 등 큰 데이터 전달 가능

단점

- 메모리 접근 필요
- 레지스터 방식보다 실행 속도가 느림
- 스택 정렬 및 관리 필요

즉, 스택을 사용하는 방식은 유연하지만 약간의 실행 시간 오버헤드가 발생한다.

[4] Caller 와 Callee

함수 호출 관계에서는 다음과 같은 용어를 사용한다.

- Caller: 함수를 호출하는 루틴
- Callee: 호출되는 루틴

예:

```
main:  
    call funcA
```

이 경우:

- main: caller
- funcA: callee

인수 전달 방식은 caller 와 callee 사이의 약속(호출 규약)에 의해 결정된다.

[12-8] System V AMD64 ABI 에서의 실제 인수 전달

앞 절에서 인수 전달의 개념(call-by-value, call-by-reference)을 살펴보았다.

이제 Linux 64 비트 환경에서 실제로 어떤 레지스터를 사용하여 인수를 전달하는지 살펴보자.

이 규약은 System V AMD64 ABI 라고 불리며,
64 비트 Linux 에서 C/C++ 컴파일러가 사용하는 표준 호출 규약이다.

따라서 어셈블리 코드와 C 코드가 서로 호출될 수 있는 이유도 이 동일한 규약을 따르기 때문이다.

[12-8-1] 정수형 인수 전달 레지스터

System V AMD64 ABI 에서는

처음 여섯 개의 정수형 또는 포인터 인수를 다음 레지스터에 저장하여 전달한다.

인수 순서	사용 레지스터
1번째	RDI
2번째	RSI
3번째	RDX
4번째	RCX
5번째	R8
6번째	R9

예시(C 코드):

```
func(a, b, c, d, e, f):
```

호출 직전 레지스터 상태는 다음과 같다.

```
RDI = a  
RSI = b  
RDX = c  
RCX = d  
R8  = e  
R9  = f
```

그 다음 call func 가 실행된다.

[12-8-2] 7 번째 인수부터는 스택 사용

여섯 개를 초과하는 인수는 스택을 사용한다.

예:

```
func(a, b, c, d, e, f, g, h):
```

전달 방식:

```
RDI = a  
RSI = b  
RDX = c  
RCX = d  
R8  = e  
R9  = f  
g → 스택  
h → 스택
```

즉,

- 레지스터가 우선
- 초과분은 스택

이라는 구조이다.

[12-8-3] 반환값(Return Value)

함수의 반환값은 기본적으로 다음 레지스터에 저장된다.

반환 타입	사용 레지스터
정수형 / 포인터	RAX
128비트 이하	RDX:RAX (필요 시)

즉, 함수가 종료될 때 결과 값은 RAX 에 넣어두면 된다.

예:

```
mov rax, 42  
ret
```

이 함수는 42 를 반환한다.

[12-8-4] call-by-reference 의 실제 구현

call-by-reference 는 특별한 메커니즘이 있는 것이 아니다.
단지 주소를 레지스터에 넣어 전달하는 것이다.

예:

```
void func(int *p);
```

호출 시:

```
RDI = p 의 주소
```

함수 내부에서는:

```
mov eax, [rdi] ; p 가 가리키는 값 읽기
```

이처럼 메모리 접근을 통해 원래 변수에 직접 접근한다.

[12-8-5] 부동소수점 인수

부동소수점(float, double)은 일반 레지스터가 아니라 XMM 레지스터를 사용한다.

인수 순서	사용 레지스터
1번째	XMM0
2번째	XMM1
3번째	XMM2
...	...

최대 8 개까지 XMM0~XMM7 사용한다.

[12-8-6] Caller-Saved vs Callee-Saved 레지스터

호출 규약에는 “누가 레지스터를 보존해야 하는가?”에 대한 약속고 포함된다.

[1] Caller-Saved(호출자가 보존)

다음 레지스터는 callee 가 마음대로 변경할 수 있다.

```
RAX  
RCX  
RDX  
RSI  
RDI  
R8-R11
```

caller 는 필요하면 call 전에 스택에 저장(push)해야 한다.

[2] Callee-Saved(피호출자가 보존)

다음 레지스터는 callee 가 변경하면 반드시 원래 값으로 복원해야 한다.

```
RBX
RBP
R12
R13
R14
R15
```

예:

```
push rbx
...
pop rbx
ret
```

[12-8-7] 전체 흐름 정리

C 코드:

```
int add(int a, int b) {
    return a + b;
}
```

실제 레지스터 동작:

호출 시:

```
RDI = a
RSI = b
```

부동소수점(float, double)은 일반 레지스터가 아니라 XMM 레지스터를 사용한다.

함수 내부:

```
add:
    mov rax, rdi
    add rax, rsi
    ret
```

복귀 시:

```
RAX = a + b
```

caller 는 RAX

[12-8-8] 전체 구조

인수 전달

- 1~6 번째 → RDI, RSI, RDX, RCX, R8, R9
- 7 번째 이후 → 스택

반환값

- 기본 → RAX

부동소수점

- XMM0~XMM7

레지스터 보존

- Caller-Saved: RAX, RCX, RDX, RSI, RDI, R8-R11
- Callee-Saved: RBX, RBP, R12-R15

[12-9] 호출 규약(Calling Convention)

함수가 호출되면 단순히 점프만 일어나는 것이 아니다.

프로그램의 실행 상태(레지스터, 스택 상태 등)가 일시적으로 변경되었다가
함수가 끝나면 다시 원래 상태로 복원되어야 한다.

이 전체 과정을 체계적으로 정의해 놓은 약속이 바로 호출 규약이다.

함수 프롤로그(Function Prologue)

프롤로그는 함수 시작 부분의 코드이다.

이 부분에서는 일반적으로 다음과 같은 작업을 수행한다.

- 필요한 레지스터 저장
- 스택 프레임 설정
- 스택 동적 지역 변수 공간 확보

즉,

함수 실행을 위한 준비 작업
을 수행하는 영역이다.

함수 에필로그(Function Epilogue)

에필로그는 함수 끝 부분의 코드이다.

이 부분에서는:

- 저장해 둔 레지스터 복원
- 스택 정리
- 원래 실행 상태 복구
- ret 실행

즉,
함수 실행으로 인해 변경된 상태를 원래대로 되돌리는 작업을 수행한다.

호출 규약의 기본 개념

핵심 개념은 매우 단순하다.

1. 프로그램 상태(특정 레지스터, 스택 상태)를 저장한다.
2. 함수 본문을 실행한다.
3. 저장했던 상태를 복원한다.

함수는 내부에서 레지스터와 스택을 자유롭게 사용할 수 있지만, 반환 시에는 약속된 상태가 유지되어야 한다.

프로로그는 상태 저장을 돕고,
에필로그는 상태 복원을 담당한다.

[12-9-1] 매개변수 전달(Parameter Passing)

함수에 매개변수를 전달하는 방식은
레지스터와 스택을 조합하여 이루어진다.

[1] 처음 여섯 개의 정수형 인수

System V AMD64 ABI 에 따르면 처음 여섯 개의 정수형(또는 포인터) 인수는 레지스터를 사용한다.

인수 번호	64-bit	32-bit	16-bit	8-bit
1	rdi	edi	di	dil
2	rsi	esi	si	sil
3	rdx	edx	dx	dl
4	rcx	ecx	cx	cl
5	r8	r8d	r8w	r8b
6	r9	r9d	r9w	r9b

이 표는 인수 크기에 따라 사용할 하위 레지스터를 보여준다.

예를 들어:

```
func(a, b, c);
```

라면 호출 직전:

```
rdi = a  
rsi = b  
rdx = c
```

[2] 일곱 번째 이후 인수

7 번째 인수부터는 스택을 통해 전달된다.

중요한 규칙

스택에 push 할 때는 역순(reverse order)으로 push 한다.

예:

```
someFunc(one, two, three, four, five, six, seven, eight, nine)
```

push 순서:

```
nine  
eight  
nine
```

역순으로 push 되는 이유는 함수 내부에서 인수를 순서대로 접근하기 위해서이다.

[3] 부동소수점 인수

float 또는 double 타입의 경우

처음 여덟 개는 다음 레지스터를 사용한다.

```
xmm0~xmm7
```

[4] 함수 호출 후 스택 정리

표준 호출 규약에 따르면 스택에 전달된 인수를 제거할 책임은 caller 에게 있다.

여러 번 pop 을 사용하는 대신, rsp 를 한 번에 조정한다.

각 인수는 8 바이트이므로:

```
add rsp, (인수 개수 × 8)
```

을 수행하면 된다.

[5] 반환 값 위치

반환 값은 크기에 따라 다음 위치에 저장된다.

반환 크기	저장 위치
byte	al
word	ax
double-word	eax
quadword	rax
floating-point	xmm0

즉, 정수 반환 값은 R 레지스터 계열을 사용한다.

rax 는 함수 내부에서 자유롭게 사용 가능하지만, 반환 직전에는 반드시 올바른 값이 설정되어 있어야 한다.

[12-9-2] 레지스터 사용(Register Usage)

호출 규약은 함수 호출 시 레지스터 사용 방식도 명시한다.

[1] 보존 레지스터(Callee-Saved Register)

다음 레지스터는 함수가 사용하려면 원래 값을 반드시 보존해야 한다.

```
rbx  
rbp  
r12  
r13  
r14  
r15
```

동작 방식:

1. 프로로그에서 push 로 저장
2. 함수 내부에서 사용
3. 에필로그에서 pop 으로 복원

[2] 보존되지 않는 레지스터(Caller-Saved)

다음 레지스터는 함수 호출 전후에 값이 유지되지 않는다.

```
rax  
r10  
r11  
rdi  
rsi  
rdx  
rcx  
r8  
r9
```

이 레지스터들은 함수 내부에서 자유롭게 사용할 수 있다.

caller 가 해당 값을 유지해야 한다면,

call 전에 직접 저장해야 한다.

[3] 레지스터 사용 요약

Register	Usage
rax	반환 값
rbx	Callee Saved
rcx	4번째 인수
rdx	3번째 인수
rsi	2번째 인수
rdi	1번째 인수
rbp	Callee Saved
rsp	스택 포인터
r8	5번째 인수
r9	6번째 인수
r10	임시
r11	임시
r12	Callee Saved
r13	Callee Saved
r14	Callee Saved
r15	Callee Saved

[4] 부동소수점 레지스터

어떠한 부동소수점 레지스터(XMM 레지스터)도 호출 전후에 자동으로 보존되지 않는다. 즉, 필요하다면 caller 가 직접 저장해야 한다.

[5] 전체 구조 정리

호출 규약은 다음을 정의한다.

- 인수를 어디에 저장하는가
- 반환 값을 어디에 저장하는가
- 어떤 레지스터를 누가 보존해야 하는가
- 스택을 누가 정리하는가
- 프로로그와 에필로그에서 무엇을 해야 하는가

이 약속 덕분에

- 어셈블리와 C 연동 가능
- 재귀 호출 가능
- 중첩 함수 호출 가능
- 프로그램 안정성 유지 가능

[12-9-3] 콜 프레임(Call Frame)

함수가 호출되면, 그 함수의 실행을 위해 스택(stack) 위에 일정한 정보들이 저장된다. 이때 스택에 구성되는 하나의 실행 단위를 콜 프레임(call frame)이라고 한다.

콜 프레임은 다음과 같은 다른 이름으로도 불린다.

- activation record
- stack frame

즉, 함수 하나가 실행되는 동안 필요한 모든 실행 정보를 담는 스택 구조라고 이해하면 된다.

[1] 콜 프레임에 포함되는 요소

표준 호출 규약에 따르면, 콜 프레임은 다음 항목들을 포함할 수 있다.

- 반환 주소(Return Address): 반드시 존재
- 보존 레지스터(Callee-saved registers): 필요한 경우
- 스택으로 전달된 인수: 필요한 경우
- 스택 기반 지역 변수: 필요한 경우

[1-1] 반환 주소(필수)

call 명령이 실행되면,
CPU는 자동으로 다음에 실행할 명령 주소(rip)를 스택에 push 한다.
이 값이 바로 반환 주소이며,
함수가 끝날 때 ret 명령을 통해 이 주소로 되돌아간다.
따라서 반환 주소는 모든 콜 프레임에 반드시 존재한다.

[2] 콜 프레임이 항상 필요한가?

모든 함수가 오나전한 콜 프레임을 필요로 하는 것은 아니다.
예를 들어, 다음 조건을 모두 만족한다면:

- 리프 함수(leaf function)이다: 다른 함수를 호출하지 않는다.
- 인수를 레지스터로만 전달한다.
- Callee-saved 레지스터를 사용하지 않는다.
- 스택 기반 지역 변수가 필요하지 않다.

이 경우 스택을 거의 건드리지 않아도 된다.
즉, 프로로그와 에필로그가 매우 단순해질 수 있다.

이런 함수는 보통 짧고 단순한 함수이다.
하지만 위 조건 중 하나라도 만족하지 않으면 정상적인 콜 프레임 구성이 필요하다.

[3] 프레임 포인터(rbp)의 역할

표준 호출 규약은 rbp 사용을 강제하지는 않는다.

최적화를 위해 컴파일러는 rbp를 생략할 수도 있다.

그러나 교육 목적이나 구조 이해를 위해 rbp를 프레임 포인터로 사용하는 방식이 가장 직관적이다.

[3-1] 기본 구조

함수 시작 시:

```
push rbp  
mov rbp, rsp
```

의미는 다음과 같다.

1. 이전 함수의 rbp 값을 저장
2. 현재 함수의 기준점(base)을 설정

이후 여러 번 push/pop을 수행하여 rsp가 변해도 rbp는 변하지 않는다.

따라서:

- 스택 인수 접근
- 지역 변수 접근

을 rbp를 기준으로 일정한 오프셋으로 접근할 수 있다.

[4] 예시: 인수 8개 + 레지스터 3개 보존

가정:

- 정수 인수 8개
- rbx, r12, r13을 사용(Callee-saved → push 필요)

그 경우 스택 구조는 다음과 같다.

〈8th Argument〉	← rbp + 24
〈7th Argument〉	← rbp + 16
Return Address	← rbp + 8
Saved rbp	← rbp
Saved rbx	
Saved r12	
Saved r13	← rsp

[4-1] 왜 +16, +24 인가?

- $rsp + 0$: 저장된 `rbp`
- $rbp + 8$: 반환 주소
- $rbp + 16$: 첫 번째 스택 인수(7 번째 전체 인수)
- $rbp + 24$: 두 번째 스택 인수(8 번째 전체 인수)

각 `push` 는 8 바이트(64 비트)이다.

[5] 콜 프레임의 핵심 개념

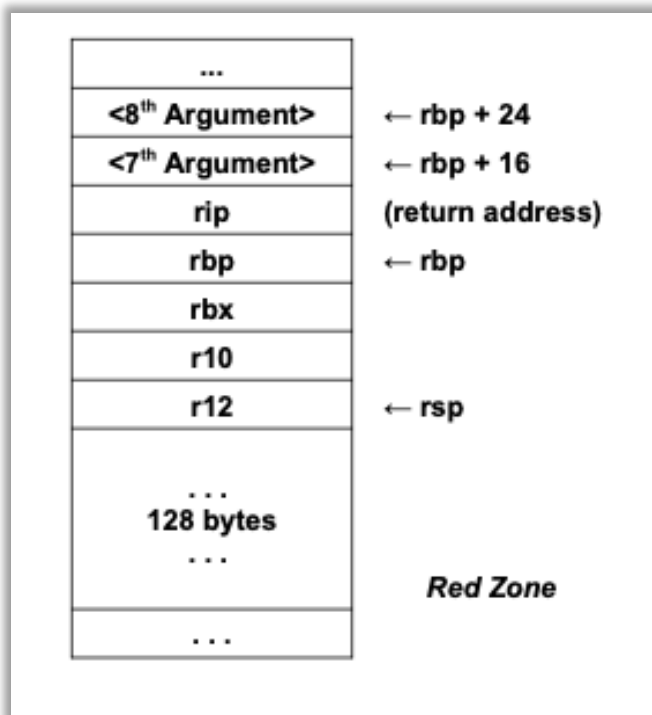
콜 프레임의 본질은 다음 한 문장으로 요약할 수 있다:

“함수 실행 전의 상태를 보존하고,
함수 실행 후 그 상태를 복원하기 위한 구조”

이를 통해 프로그램은 함수 호출이 중첩되어도 정확히 이전 상태로 복귀할 수 있다.

[12-9-4] Red Zone

Linux x86-64 표준 호출 규약(System V ABI)에는
Red Zone 이라는 특별한 영역이 존재한다.



[1] Red Zone 이란?

`rsp` 아래의 128 바이트 영역은 함수가 임시로 사용할 수 있도록 예약되어 있다.

중요한 점:

- 이 영역은 인터럽트나 시그널 핸들러에 의해 덮어쓰이지 않는다.
- 따라서 스택 포인터를 조정하지 않고도 사용 가능하다.

[2] 존재하는 이유

목적은 컴파일러 최적화이다.

작은 지역 변수의 경우:

```
sub rsp, 32
```

와 같은 스택 조정 없이

```
[rsp - 8]
```

처럼 바로 사용할 수 있다.

즉,

- push/pop 감소
- 명령어 수 감소
- 실행 속도 향상

이라는 이점이 있다.

[3] 어셈블리 프로그래머에게 중요한가?

일반적인 C 컴파일러 최적화에서는 유용하지만,

어셈블리를 직접 작성할 경우:

- 인터럽트 가능성
- 커널 코드 작성 여부
- 스택 정렬 요구

등을 고려해야 한다.

특히 커널 코드에서는 Red Zone 을 사용할 수 없다.

[4] 전체 흐름 정리

1. 함수 호출 → 반환 주소 push
2. 필요하면 프로로그 실행
 - rbp 저장
 - rbp 설정
 - Callee-saved 저장
 - 지역 변수 공간 확보
3. 함수 실행
4. 에필로그 실행
 - 저장 레지스터 복원
 - rbp 복원
5. ret → 반환 주소로 복귀

[12-10] 예제: 통계함수 1 - 리프 함수

이 예제는 배열의 합(sum)과 평균(average)을 계산하는 간단한 void 함수를 다룬다. 특히 이 함수는 리프 함수이다.

리프함수는 다른 함수를 호출하지 않는 함수를 의미한다.

따라서 스택 사용이 최소화되며, 콜 프레임도 매우 단순한 형태를 가진다.

[12-10-1] 고급 언어(HLL) 호출 형태

C/C++에서의 함수 호출은 다음과 같다.

```
stats1(arr, len, &sum, &avg);
```

[1] 인수 전달 방식 분석

인수	의미	전달 방식
arr	배열의 시작 주소	call-by-reference
len	배열 길이	call-by-value
&sum	합을 저장할 변수의 주소	call-by-reference
&ave	평균을 저장할 변수의 주소	call-by-reference

[2] 중요한점

- 배열은 항상 주소 전달
- 결과를 저장할 변수는 아직 값이 없으므로 주소 전달
- len 만 값 전달

또한:

- arr, sum, agv → unsigned double-word(32-bit 정수)
- len → 문맥상 unsigned 가 적절

[12-10-2] Caller(호출자)

이 예제는 인수가 4 개이므로, 모두 레지스터를 통해 전달된다.

x84-64 System V 호출 규약에서 정수 인수는 다음 순서를 따른다.

1. rdi
2. rsi
3. rdx
4. rcx

[1] 호출 코드

```
; stats1(arr, len, &sum, &avg);
```

```
mov rcx, avg    ; 4th arg, addr of avg
```

```
mov rdx, sum    ; 3th arg, addr of sum
```

```
mov esi, dword [len] ; 2nd arg, value of len
```

```
mov rdi, arr    ; 1st arg, addr of arr
```

[1-1] 설정 순서

레지스터 설정 순서에는 제한이 없다.

이 예제는 이후 확장을 대비해 역순으로 설정하였다.

[2] esi 를 사용하는 이유

```
mov esi, dword [len]
```

32 비트 레지스터(esi)에 값을 넣으면, 상위 32 비트는 자동으로 0 으로 된다.

따라서:

```
- len 이 unsigned 라면 rsi 전체가 올바르게 설정된다.
```

이는 x86-64 의 중요한 특징 중 하나이다.

[3] 반환 값은?

이 함수는 void 함수이다. 따라서 rax 에 반환 값을 설정하지 않는다.

만약 값을 반환하는 함수였다면:

```
- byte: al
```

```
- word: ax
```

```
- dword: eax
```

```
- qword: rax
```

에 저장되었을 것이다.

[12-10-3] Callee(피호출자)

피호출자(stats1)는 표준 호출 규약에 따라

1. 필요한 레지스터를 저장(프로로그)
2. 본래 기능 수행
3. 저장한 레지스터 복원(에필로그)

을 수행해야 한다.

[1] 수행해야 할 작업

이 함수는:

- 배열의 합 계산
- 정수 평균 계산
- sum 과 avg 에 결과 저장

을 수행한다.

[2] 전체 코드

```
global stats1
stats1:
    push r12        ; prologue
    mov r12, 0      ; counter / index
    mov rax, 0       ; running sum

sumLoop:
    add eax, dword [rdi+r12*4] ; sum += arr[i]
    inc r12
    cmp r12, rsi
    jl sumLoop      ; 피연산자 1 < 피연산자 2

    mov dword [rdx], eax ; return sum
    cdq
    idiv esi          ; compute average
    mov dword [rcx], eax ; return ave

    pop r12           ; epilogue
    ret
```

[2-1] 코드 해설

```
push r12
```

r12 는 Callee-Saved 레지스터이다.

따라서,
사용하기 전에 저장해야 하고,
함수 종료 전에 복원해야 한다.

루프 동작

```
add eax, dword [rdi+r12*4]
```

- rdi: 배열 시작 주소
- r12: 인덱스
- *4: int(4 바이트) 크기
- eax: 누적합

즉, $\text{sum} += \text{arr}[i]$ 과 동일하다.

평균 계산

```
cdq  
idiv esi
```

cdq 의 역할은 eax 를 edx:eax 형태로 64 비트 형태로 확장한다.

```
idiv esi
```

```
(edx:eax) / esi
```

- 몫: eax
- 나머지: edx

즉,
 $\text{average} = \text{sum} / \text{len}$

결과 저장

```
mov dword [rdx], eax  
mov dword [rcx], eax
```

rdx 와 rcx 는 주소이므로 해당 메모리 위치에 값을 저장한다.
이것이 call-by-reference 의 실제 구현이다.

[12-11] 예제: 통계함수 2 - 리프 함수

이 확장된 예제는 다음 값을 모두 계산하는 void 함수이다.

- 최소값(min)
- 중앙값 1(med1)
- 중앙값 2(med2)
- 최대값(max)
- 합(sum)
- 평균(ave)

호출 형태는 다음과 같다.

```
stats2(arr, len, &min, &med1, &med2, &max, &sum, &ave);
```

함수의 특징

- 배열은 오름차순으로 정렬되어 있음
- 중앙값은 “가운데 값”으로 정의
- 길이가 짝수일 경우 가운데 값 두 개 반환
- 길이가 홀수일 경우 하나의 가운데 값을 med1, med2 에 모두 저장

인수 전달 방식 분석

인수	전달 방식
arr	call-by-reference
len	call-by-value
min	call-by-reference
med1	call-by-reference
med2	call-by-reference
max	call-by-reference
sum	call-by-reference
ave	call-by-reference

모두 signed double-word 정수이며,
len 만 unsigned 로 보는 것이 문맥상 적절하다.

[12-11-1] Caller(호출자)

인수가 8 개이므로:

- 처음 6 개: 레지스터
- 7 번째, 8 번째: 스택

[1] 전달 순서(System V ABI):

1. rdi
2. rsi
3. rdx
4. rcx
5. r8
6. r9
7. stack
8. stack

[2] 호출 코드

```
push avg
push sum
mov r9, max
mov r8, med2
mov rcx, med1
mov rdx, min
mov esi, dword [len]
mov rdi, arr

call stats2
add rsp, 16
```

[3] 왜 역순 push 인가?

스택은 아래로 성장한다. 따라서 가장 마지막 인수부터 push 해야

[rbp+16] : 7 번째 인수
[rbp+24]: 8 번째 인수

형태가 유지된다.

[4] call 이후 스택 정리

```
add rsp, 16
```

- 2 개 인수
- 각 8 바이트

16 바이트 제거하며 System V ABI에서는 caller 가 스택 인수를 정리한다.

[12-11-2] Callee(피호출자)

이 함수는 비리프 함수이다.

비리프 함수의 특징

- 더 많은 레지스터 사용
- 스택 인수 접근
- 프레임 포인터 사용
- 더 완전한 콜 프레임 필요

[1] 프로로그

```
push rbp
mov rbp, rsp
push r12
```

왜 필요한가?

- rbp: 프레임 포인터 설정
- r12: Callee-Saved 레지스터이므로 보존

[2] 최소값과 최대값

배열이 정렬되어 있으므로:

- arr[0]: 최소값
- arr[len-1]: 최대값

```
mov eax, [rdi]
mov [rdx], eax

mov r12, rsi
dec r12
mov eax, [rdi+r12*4]
mov [r9], eax
```

정렬되어 있다는 가정 덕분에 반복문이 필요 없다.

[3] 중앙값 계산

```
len / 2
mov rax, rsi
mov rdx, 0
mov r12, 2
div r12
```

- rax: 몫
- rdx: 나머지

[3-1] 홀수 길이(나머지가 0 이 아닐 때)

```
arr[len/2]
```

을 med1, med2 에 모두 저장

[3-2] 짝수 길이(나머지가 0 일 때)

```
arr[len/2]
```

```
arr[len/2 - 1]
```

[4] 합 계산

리프 함수 stats1 과 동일한 루프 구조:

```
mov r12, 0
```

```
mov rax, 0
```

루프 반복:

```
sum += arr[i]
```

[5] 스택 인수 접근

sum 과 ave 는 스택에 있다.

[5-1] 7 번째 인수

```
mov r12, qword[rbp+16]
```

```
mov dword [r12], eax
```

왜 두 단계인가?

[rbp+16]에는 “주소”가 들어 있다.

따라서

1. 주소를 꺼내고

2. 그 주소에 저장

[5-2] 혼한 오류

```
mov dword [rbp+16], eax
```

이 코드는

- 스택에 저장된 주소 자체를 덮어쓴다.

- 실제 sum 변수를 변경하지 않는다.

이 것은 매우 자주 발생하는 실수이다.

[6] 평균 계산

```
cdq
idiv rsi
```

- `edx:eax / rsi`
- 몫: `eax`

이후:

```
mov r12, [rbp+24]
mov [r12], eax
```

[7] 에필로그

```
pop r12
pop rbp
ret
```

push 의 역순으로 pop 해야 한다.

[8] 콜 프레임 구조

이 함수의 스택 구조는 다음과 같다.

```
<8th Argument>   ← rbp + 24
<7th Argument>   ← rbp + 16
Return Address    ← rbp + 8
Saved rbp         ← rbp
Saved r12
← rsp
```

리프 함수보다 더 복잡하다.

[12-12] 스택 기반 지역 변수(Stack-Based Local Variables)

함수에서 지역 변수가 필요하면 그 변수는 스택(stack)에 할당된다.
할당 방법은 단순하다.

- `rsp` 를 감소시켜 공간을 확보한다.

왜 감소인가?

- 스택은 메모리에서 아래 방향(낮은 주소 방향)으로 성장하기 때문이다.

[1] 지역 변수의 생명 주기

- 함수 시작: 스택에 공간 할당
- 함수 종료: 스택 원상 복구
- 함수가 끝나면 지역 변수는 자동 소멸

즉, 지역 변수는 함수 호출 동안만 존재한다.

[2] 예제 확장: mode 계산

가정:

- 배열 값은 0~99
- 최빈값(mode)을 계산

필요한 지역 변수

- count: double-word(4bytes)
- tmpArr[100]: $100 \times 4 = 400$ bytes

총 필요 공간

$$4 + 400 = 404 \text{ bytes}$$

[3] 스택 공간 할당

프롤로그

```
push rbp
mov rbp, rsp
sub rsp, 404
push rbx
push r12
```

[3-1] 중요한 점

왜 `sub rsp, 404` 인가

404 바이트를 확보해야 하므로 `rsp` 를 404 만큼 감소시킨다.

왜 저장 레지스터를 그 다음에 `push` 하는가

현재 스택 구조:

```
| 저장 r12 |
| 저장 rbx |
| count   |
| tmpArr  |
| rbp     |
| ret addr |
```

저장 레지스터는 지역 변수 아래에 위치하게 된다.

[4] 콜 프레임 구조

〈value of len〉	$\leftarrow \text{rbp} + 24$
〈addr of list〉	$\leftarrow \text{rbp} + 16$
Return Address	$\leftarrow \text{rbp} + 8$
Saved rbp	$\leftarrow \text{rbp}$
tmpArr[99]	
...	
tmpArr[0]	$\leftarrow \text{rbp} - 400$
count	$\leftarrow \text{rbp} - 404$
Saved rbx	
Saved r12	$\leftarrow \text{rsp}$

[5] 지역 변수 접근 방식

지역 변수는 항상:

$\text{rbp} - \text{offset}$

형태로 접근한다.

[5-1] count 초기화

`mov dword [rbp-404], 0`

이것이 count의 정확한 위치이다.

[5-2] count 증가

`inc dword [rbp-404]`

[6] 배열 접근

tmpArr의 시작 주소는:

$\text{rbp} - 400$

주소를 레지스터에 담기 위해 lea 사용:

`lea rbx, [rbp-400]`

주의:

`lea rbx, dword [rbp-400]`

여기서 dword는 의미가 없다.

주소는 항상 64비트이기 때문이다.

정확한 형태는:

`lea rbx, [rbp-400]`

[7] 배열 초기화 루프 분석

```
lea rbx, [rbp-400]
mov r12, 0
```

루프:

```
inc r12
cmp r12, 100
jl zeroLoop
mov dword [rbx+r12*4], 0
```

이 예제는 교육용 코드이며 완전하지 않다.(실제로는 mov 명령이 jl 보다 위에 있어야 한다.)

정상적이라면:

```
zeroLoop:
    mov dword [rbx+r12*4], 0
    inc r12
    cmp r12, 100
    jl zeroLoop
```

[8] 에필로그

```
pop r12
pop rbx
mov rsp, rbp
pop rbp
ret
```

[8-1] 왜 mov rsp, rbp 가 좋은가

대안은:

```
add rsp, 404
```

하지만 이 방식은:

- 지역 변수 크기가 바뀌면 에필로그도 수정해야 한다.

반면:

```
mov rsp, rbp
```

은 항상 정확한 위치로 복원한다.

[9] 매우 중요한 개념

지역 변수는 자동 초기화되지 않는다.

```
sub rsp, 404
```

은 단지 공간만 확보한다.

메모리 내용은 이전 함수가 남긴 쓰레기 값일 수 있다.

따라서 반드시 직접 초기화해야 한다.

[10] 전체 흐름

함수 진입:

1. 이전 rbp 저장
2. 새 프레임 설정
3. 지역 변수 공간 확보
4. Callee-Saved 저장

함수 종료:

1. Callee-Saved 복원
2. rsp 를 rbp 로 복구
3. rbp 복원
4. ret

[12-13] 요약 - 표준 호출 규약 정리

이 절은 x86-64 System V ABI 표준 호출 규약의 핵심 요구사항을 정리한 것이다.

[12-13-1] 호출자 동작

[1] 인수 전달 규칙

처음 6 개의 정수 인수: 레지스터

순서	레지스터
1	rdi
2	rsi
3	rdx
4	rcx
5	r8
6	r9

7 번째 이후 인수: 스택

- 오른쪽 → 왼쪽 순서로 push
- quadword(8 바이트) 단위로 push

예:

```
func(a, b, c, d, e, f, g, h);
```

push 순서:

```
push h  
push g
```

[2] 함수 호출

```
call func
```

call 명령은

- 반환 주소(rip)를 스택에 push
- 제어를 피호출자에게 전달

[3] 스택 인수 정리

System V 규약에서는:

Caller 가 스택 인수를 제거한다.

```
add rsp, argCount*8
```

예:

```
add rsp, 16
```

[12-13-2] 피호출자 동작

[1] 함수 프로로그

스택 인수가 있는 경우

```
push rbp  
mov rbp, rsp
```

rbp 를 프레임 포인터로 설정한다.

[1-1] rbp 기준 스택 구조

위치	내용
[rbp]	이전 rbp
[rbp+8]	반환 주소
[rbp+16]	첫 스택 인수
[rbp+24]	두 번째 스택 인수

[1-2] 지역 변수 할당

sub rsp, N

지역 변수는:

[rbp - offset]

형태로 접근한다.

[1-3] 반환값 설정

크기	반환 레지스터
byte	al
word	ax
dword	eax
qword	rax
float/double	xmm0

[1-4] 반드시 보존해야 하는 레지스터

Callee-Saved

rbx
r12
r13
r14
r15
rbp

이 레지스터를 변경하면

push reg
...
pop reg

으로 복원해야 한다.

[2] 함수 실행

- 본래 기능 수행
- 인수 사용
- 지역 변수 사용
- 필요 시 다른 함수 호출

[3] 함수 에필로그

[3-1] 저장한 레지스터 복원

pop r15
pop r14

[3-2] 지역 변수 제거

권장 방식

```
mov rsp, rbp
```

(= sub 했던 공간을 한 번에 복원)

[3-3] 이전 rbp 복원

```
pop rbp
```

[3-4] 호출자에게 복귀

```
ret
```

- 스택에서 반환 주소 pop
- 해당 위치로 점프

Chapter13. 시스템 서비스(System Services)

왜 시스템 서비스가 필요한가?

일반 응용 프로그램은:

- 콘솔에 출력
- 키보드 입력
- 파일 열기/읽기/쓰기
- 시간 얻기
- 메모리 할당

같은 작업을 직접 수행할 수 없다.

왜냐하면 이런 작업은 하드웨어 접근 권한(특권, privilege)이 필요하기 때문이다.

CPU에는 두 가지 모드가 있다.

- User Mode: 우리가 작성한 프로그램
- Kernel Mode: 운영체제만 실행 가능

일반 프로그램은 Kernel Mode로 직접 들어갈 수 없다.

그래서 필요한 것이 바로 System call(시스템 호출)이다.

System Call이란?

실행 중인 프로세스와 운영체제 사이의 공식 인터페이스

쉽게 말하면:

프로그램 → (요청) → 운영체제

예:

- “파일 열어줘”
- “화면에 출력해줘”
- “메모리 더 줘”

운영체제가 대신 수행한다.

[13-1] 시스템 서비스 호출(Calling System Services)

[1] 함수 호출과 비슷하다

일반 함수 호출

call myFunction

시스템 호출:

syscall

차이점:

일반 함수	시스템 호출
같은 프로그램 내부	OS 내부 코드
같은 권한 수준	커널 모드로 전환
ret으로 복귀	syscall 종료 후 복귀

[2] 호출 절차

[2-1] rax 에 “시스템 호출 번호” 저장

각 시스템 호출은 고유 번호를 가진다.

예(Linux 기준):

서비스	rax 값
write	1
read	0
exit	60

이 값은 운영체제가 정해놓은 상수다.

[2-2] 인수를 레지스터에 배치

시스템 호출은 최대 6 개 인수만 사용

순서	레지스터
1	rdi
2	rsi
3	rdx
4	r10
5	r8
6	r9

일반 함수 호출과 거의 같지만 4 번째 인수는 rcx 가 아니라 r10 이다.

syscall 명령이 rcx 를 내부적으로 사용하기 때문이다.

[2-3] syscall 실행

syscall

이 명령이 실행되면:

1. CPU 가 Kernel Mode 로 전환
2. 운영체제가 rax 값 확인
3. 해당 서비스 실행
4. 결과를 rax 에 저장
5. 다시 User Mode 로 복귀

[3] 예제: 화면에 출력하기(Linux)

```
section .data
msg db "Hello", 10
len equ $ - msg

section .text
global _start

_start:

    mov rax, 1      ; sys_write
    mov rdi, 1      ; stdout
    mov rsi, msg    ; 주소
    mov rdx, len    ; 길이
    syscall

    mov rax, 60     ; sys_exit
    xor rdi, rdi    ; exit code 0
    syscall
```

[4] 시스템 호출 vs 함수 호출 차이 핵심

항목	함수 호출	시스템 호출
명령어	call	syscall
권한 전환	없음	user → kernel
인수	6개 + 스택	최대 6개
호출 번호	필요 없음	rax 필수

[13-2] 개행 문자

[1] newline 이란?

커서를 다음 줄의 시작으로 이동

C에서는:

“Wn”

ASCII 값:

10(0x0A)

이를 LF(Line Feed)라고 한다.

[2] Unix/Linux vs Windows 차이

[2-1] Linux / Unix

newLine = LF(0x0A)

[2-2] Windows

newline = CR + LF

문자	값
CR (Carriage Return)	13 (0x0D)
LF (Line Feed)	10 (0x0A)

즉:

0x0D 0x0A

[2-3] 왜 옛날 Notepad 에서 줄이 안 나뉘었는가?

Linux 파일은 LF 만 있음

Windows 옛 Notepad 는 CRLF 를 기대 → LF 만 있으면 줄 구분 못함

그렇기 때문에 전부 한 줄 처럼 보인다.

[13-3] 콘솔 문자(Console Output)

[1] 개요

콘솔에 문자열을 출력하는 시스템 서비스는 write 시스템 호출

Linux/Unix 환경에서 write 는

파일 디스크립터(File Descriptor)를 통해 출력 대상에 데이터를 보낸다.

[2] File Descriptor 개념

운영체제는 입출력을 파일처럼 취급한다.

이미 열려 있는 기본 파일 디스크립터:

이름	값	의미
STDIN	0	표준 입력 (키보드)
STDOUT	1	표준 출력 (콘솔 화면)
STDERR	2	표준 오류 출력

따라서 콘솔에 출력하려면:

rdi = 1 (STDOUT)

을 사용한다.

[3] write 시스템 호출 규약

Linux x86-64 기준 레지스터 사용

레지스터	의미
rax	시스템 호출 번호 (1 = write)
rdi	파일 디스크립터
rsi	출력할 데이터의 주소
rdx	출력할 바이트 수

[4] 기본 예제

상수 정의

```
STDOUT equ 1
SYS_write equ 1
```

데이터 정의

```
msg db "Hello World"
msgLen dq 11
```

출력 코드

```
mov rax, SYS_write
mov rdi, STDOUT
mov rsi, msg
mov rdx, qword [msgLen]
syscall
```

[5] 중요한 점

운영체제는

- 문자열이 NULL로 끝나는지 검사하지 않음
- 지정된 길이만큼 그대로 출력

즉, rdx가 100이면 100 바이트를 출력하려 시도

- 메모리 접근 오류 가능

[13-3-1] 예제: Console Output

```
section .data
LF equ 10          ; line feed
NULL equ 10        ; end of string
TRUE equ 1
FALSE equ 0
EXIT_SUCCESS equ 0 ; success code
STDIN equ 0         ; standard input
STDOUT equ 1        ; standard output
STDERR equ 2        ; standard error
SYS_read equ 0       ; read
SYS_write equ 1      ; write
SYS_open equ 2       ; file open
SYS_close equ 3      ; file close
SYS_fork equ 57      ; fork
SYS_exit equ 60      ; terminate
SYS_create equ 85     ; file open/create
SYS_time equ 201     ; get time
```

```
message1 db "Hello World.", LF, NULL
message2 db "Enter Answer: ", NULL
newLine db LF, NULL
```

```
section .text
global _start
```

```
_start:
; ----
; Display first message
mov rdi, message1
call printString

; ----
; Display second message and the newline
mov rdi, message2
call printString

; ----
; Example program done.

exampleDone:
```

```

mov rax, SYS_exit
mov rdi, EXIT_SUCCESS
syscall

global printString
printString:

push rbx

; ----
; Count character in string.

mov rbx, rdi
mov rdx, 0

strCountLoop:
    cmp byte [rbx], NULL
    je strCountDone
    inc rdx
    inc rbx
    jmp strCountLoop

strCountDone:
    cmp rdx, 0
    je prtDone

; ----
; Call OS to output string.
mov rax, SYS_write    ; system code for write()
mov rsi, rdi          ; address of chars to write
mov rdi, STDOUT       ; standard out
; RDX = count to write, set above

syscall               ; system call

; ----
; String printed, return to calling routine.

prtDone:
pop rbx
ret

```

프로그램 목적

- 문자열 2 개 출력
- 하나는 개행 포함
- 하나는 개행 미포함
- NULL-terminated 문자열 사용
- 별도 출력 함수 구현

[1] 데이터 영역

```
message1 db "Hello World.", LF, NULL
message2 db "Enter Answer:", NULL
newLine db LF, NULL
```

특징

- message1: LF 포함
- message2: 개행 없음
- newLine: 단독 LF

[2] 프로그램 흐름

```
_start:
    mov rdi, message1
    call printString

    mov rdi, message2
    call printString

    mov rdi, newLine
    call printString
```

출력 결과:

```
Hello World.
Enter Answer:
```

[3] printString 함수 분석

[3-1] 목적

NULL-terminated 문자열을

1. 길이 계산
2. write 호출

[3-2] 인수

rdi = 문자열 주소

[3-3] 반환값

없음

[3-4] 함수 구조

Prologue

push rbx

rbx 는 callee_saved 레지스터이므로 저장

문자열 길이 계산

mov rbx, rdi

mov rdx, 0

rdx = 길이 카운터

[3-5] 루프

strCountLoop:

 cmp byte [rbx], NULL

 je strCoutDone

 inc rdx

 inc rbx

 jmp strCountLoop

동작:

- 현재 문자가 NULL 인지 검사
- NULL 아니면 길이 증가
- 포인터 증가

[3-6] 길이 0 검사

cmp rdx, 0

je prtDone

빈 문자열이면 출력하지 않음

[3-7] write 시스템 호출

mov rax, SYS_write

mov rsi, rdi

mov rdi, STDOUT

syscall

- rdx 는 이미 길이 값 보유
- rsi = 문자열 주소

- rdi = 출력 대상

[3-7] Epilogue

```
pop rbx  
ret
```

[4] 전체 구조 요약

[4-1] main(_start)

- 문자열 주소 rdi 에 넣음
- printString 호출
- 종료 시 SYS_exit 호출

[4-2] printString

1. rbx 저장
2. NULL 까지 길이 계산
3. write 호출
4. rbx 복원
5. ret

[5] newline(LF) 동작

Linux 에서 newline 은:

```
LF = 10(0x0A)
```

message1 에 LF 가 포함되어 있으므로:

```
Hello World.
```

출력 후 커서가 다음 줄로 이동

[6] 설계 관점에서 이 예제가 좋은 이유

이 예제는 다음 개념을 모두 포함한다.

- 시스템 호출 사용
- 파일 디스크립터 개념
- NULL-terminated 문자열
- 문자열 길이 계산 알고리즘
- 루프 사용
- Callee-Saved 레지스터 보존
- 함수 호출 구조
- LF 사용

[7] 핵심 개념

write 시스템 호출

```
rax = 1
rdi = fd
rsi = buffer
rdx = length
syscall
```

[13-4] 콘솔 입력(Console Input)

[1] 개요

콘솔(키보드)에서 문자를 읽기 위해 사용하는 서비스는 read 시스템 호출이다.
고급 언어와 마찬가지로, 콘솔 입력은 표준 입력(STDIN)으로부터 읽는다.

STDIN 은

파일 디스크립터 0

으로 정의되어 있으며, 이미 열려 있는 기본 입력 스트림이다.
따라서 어셈블리 프로그램에서도 별도의 파일 열기 없이 바로 사용할 수 있다.

[2] 키보드 입력의 특수성

콘솔 입력은 단순한 파일 읽기보다 복잡하다.
그이유는

- 사용자가 입력을 언제 끝낼지 알 수 없음
- Enter 키가 눌러야 입력이 완료됨
- 입력 길이가 사전에 정확히 정해지지 않음

[3] read 시스템 호출의 구조

read 시스템 호출은 다음과 같이 구성된다 .

Register	SYS_read
rax	호출 코드 = 0
rdi	입력 위치 (STDIN = 0)
rsi	읽은 문자를 저장할 주소
rdx	읽을 최대 문자 수

기본 선언 예

```
STDIN    equ 0
SYS_read equ 0
inChar   db 0
```

[4] 문자 하나 읽기 예

```
mov rax, SYS_read
mov rdi, STDIN
mov rsi, inChar
mov rdx, 1
syscall
```

이 코드는:

- 키보드로부터 1 바이트 읽고 그 값을 inChar 에 저장한다.

[5] 읽기 길이를 반드시 지정해야 하는 이유

read 는 다음과 같은 특성을 가진다.

- 반드시 읽을 최대 길이를 지정해야 함
- 운영체제는 문자열 개념을 모름
- NULL 종료 자동 처리 없음

즉, 프로그램이 직접

- 버퍼 공간 확보
- 문자열 종료 처리
- 개행 문자 처리

를 수행해야 한다.

[6] 입력 길이에 따른 동작

[6-1] 요청 길이보다 많이 입력한 경우

예:

- 10 바이트 읽기 요청
- 사용자가 15 자 입력

결과:

- 처음 10 바이트만 읽힘
- 나머지는 입력 스트림에 남음

큰 문제는 아니지만, 다음 read 호출에 영향을 줄 수 있다.

[6-2] 요청 길이보다 적게 입력한 경우

예:

사용자 입력

HELLO + Enter

실제 입력 스트림:

HELLOLF

총 6 바이트가 읽힌다.
즉, newline(LF)도 입력에 포함된다.

[7] 파일 리디렉션 문제

파일 입력이 리디렉션된 경우:

예:

파일 내용

```
HELLO
WORLD
```

10 바이트 읽기 요청 시:

```
HELLOWnWORLD
```

처럼 줄 경계를 무시하고 바이트 단위로 읽는다.
이는 대화형 입력과 다르게 동작할 수 있다.

[8] 해결 방법: 한 문자씩 읽기

대화형 입력에서는 일반적으로 LF(Enter)가 나올 때까지 한 번에 한 문자씩 읽는다.

이 방법의 장점:

- 정확히 한 줄만 읽을 수 있음
- 버퍼 오버플로우 방지 가능
- 초과 입력을 안전하게 제거 가능

[13-4-1] 예제: Console Input

[1] 목적:

이 프로그램은

- 사용자로부터 최대 50 자를 읽는다.
- LF 전까지 읽는다.
- NULL 종료 문자를 추가한다.
- 입력된 문자열을 다시 출력하여 확인한다.

```
section .data
; ----
; Define standard constants.
LF equ 10
NULL equ 0
TRUE equ 1
FALSE equ 0
EXIT_SUCCESS equ 0
```

```

STDIN equ 0
STDOUT equ 1
STDERR equ 2

SYS_read equ 0
SYS_write equ 1
SYS_open equ 2
SYS_close equ 3
SYS_fork equ 57
SYS_exit equ 60
SYS_create equ 85
SYS_time equ 201

; ----
; Define some strings.
STRLEN equ 50
pmpt db "Enter Text: ", NULL
newLine db LF, NULL

section .bss
chr resb 1
inLine resb STRLEN+2 ; total of 52

section .text
global _start:

_start:
; ----
; Display prompt
mov rdi, pmpt
call printString

; ----
; Read character from user (one at a time)
mov rbx, inLine
mov r12, 0

readCharacters:
mov rax, SYS_read
mov rdi, STDIN

```

```

lea rsi, byte [chr]
mv rdx, 1
syscall

mov al, byte [chr]
cmp al, LF
je readDone

jnc r12
cmp r12, STRLEN
jae readCharaters

mov byte [rbx], al
inc rbx
imp readCharacters

readDone:
mov byte [rbx], NULL

; ----
; Output the line to verify successful read

mov rdi, inLine
call printString

; ----
; Example done.
exampleDone:
mov rax, SYS_exit
mov rdi, EXIT_SUCCESS
syscall

; ----
global printString
printString:
push rbx;

mov rbx, rdi
mov rdx, 0

strCountLoop:

```

```

        cmp byte [rbx], NULL
        je strCountDone
        inc rdx
        inc rbx
        jmp strCountLoop

strCountDone:
        cmp rdx, 0
        je prtDone

mov rax, SYS_write
mov rsi, rdi
mov rdi, STDOUT
syscall

prtDone:
pop rbx
ret

```

[2] 데이터 정의

```

STRLEN equ 50
pmpt db "Enter Text: ", NULL
newLine db LF, NULL

```

[3] BSS 영역

```

chr resb 1
inLine resb STRLEN+2

```

왜 STRLEN+2 인가?

- 50 문자 입력
 - LF 문자 1 개
 - NULL 종료 문자 1 개
- 총 52 바이트 필요

[4] 프로그램 흐름

[4-1] 프롬프트 출력

```

mov rdi, pmpt
call printString

```


[4-2] 초기 설정

```
mov rbx, inLine  
mov r12, 0
```

- rbx: 문자열 저장 위치
- r12: 현재 저장된 문자 수

[4-3] 입력 루프

한 문자 읽기

```
mov rax, SYS_read  
mov rdi, STDIN  
lea rsi, byte [chr]  
mov rdx, 1  
syscall
```

LF 검사

```
mov al, byte [chr]  
cmp al, LF  
je readDone
```

Enter 가 입력되면 종료

길이 검사

```
inc r12  
cmp r12, STRLNE  
jae readCharacters
```

STRLNE 을 초과하면

- 저장은 하지 않음
 - 그러나 계속 읽어 초과 입력 제거
- 이 설계는 매우 중요하다.

입력 완료 처리

```
readDone:  
mov byte [rbx], NULL
```

문자열 끝에 NULL 추가

입력 확인 출력

```
mov rdi, inLine  
call printString
```

종료

```
mov rax, SYS_exit
mov rdi, EXIT_SUCCESS
syscall
```

[9] 초과 입력 처리의 중요성

만약 50 자 도달 시 완전히 읽기를 중단한다면

- 사용자가 더 입력한 문자가
- 입력 버퍼에 남아
- 다음 read 호출에 영향을 준다.

이 예제는 “초과 문자를 읽기는 하지만 저장하지 않음”으로써 입력 스트림을 정리한다.

[13-5] 파일 열기 작업(File Open Operations)

read 및 write 와 같은 파일 작업을 수행하려면 반드시 먼저 파일을 열어야 한다.

운영체제는 파일에 접근할 때 단순히 “파일 이름”만으로 작업하지 않는다.

파일의 현재 상태, 접근 권한, 읽기/쓰기 위치 등 내부 관리 정보가 필요하다.

파일 열기 작업에는 두 가지가 있다.

- open
- open/create

이 두 작업은 다음 절에서 각각 설명한다.

[1] 왜 파일을 먼저 열어야 하는가?

파일을 열면 운영체제는 다음 정보를 관리한다.

- 파일의 전체 상태
- 현재 읽기/쓰기 위치(file offset)
- 접근 권한 정보
- 파일이 열려 있는지 여부

이 정보는 운영체제 내부 자료구조인 File Control Block(FCB)에 저장된다.

즉, 파일 디스크립터(file descriptor)는 운영체제가 올바른 FCB 를 참조하기 위한 핸들 역할을 한다.

[2] 파일 디스크립터(File Descriptor)

파일 열기 작업이 성공하면 운영체제는 파일 디스크립터를 반환한다.

- 이는 정수 값이다.
- 이후 read, write, close 등의 작업에서 반드시 필요하다.
- 프로그래머가 직접 저장하고 관리해야 한다.

파일 열기가 실패하면 rax 레지스터에 음수 값(< 0)이 반환된다. 이 값은 오류 코드이다.

[13-5-1] 파일 열기(File Open)

open 은 이미 존재하는 파일을 여는 작업이다.
파일이 존재하지 않으면 오류가 발생한다.

[1] 접근 모드 지정

파일 open 은 반드시 접근 모드를 지정해야 한다.
다음 중 하나는 반드시 포함되어야 한다.

- Read-Only Access: O_RDONLY
- Write-Only Access: O_WRONLY
- Read/Write Access: O_RDWR

이 중 하나는 필수이다.

추가 모드는 OR 연산(|)으로 결합 가능하다.

[2] 시스템 호출 인수

Register	내용
rax	호출 코드 = SYS_open (2)
rdi	NULL 종료 파일 이름 문자열 주소
rsi	파일 접근 모드 플래그

```
SYS_open equ 2
O_RDONLY equ 000000q
O_WRONLY equ 000001q
O_RDWR equ 000002q
```

[2-1] q 접미사란?

- 8 진수(octal, base 8)를 의미
- Linux 파일 권한 표기 방식과 동일

[3] 반환 값

시스템 호출 후:

- 성공: rax = 파일 디스크립터
- 실패: rax < 0 (오류 코드)

일반적인 오류 예

- 파일을 찾을 수 없음
- 접근 권한 없음
- 잘못된 파일 이름

[13-5-2] 파일 열기/생성(File Open/Create)

open/create 는 파일을 생성하는 작업이다.

동작 방식

- 파일이 존재하지 않으면: 새로 생성
 - 파일이 이미 존재하면: 기존 파일 삭제 후 새로 생성(기존 내용 모두 사라짐)
- 즉, 덮어쓰기(Overwrite) 동작이다.

[1] 접근 모드 + 파일 권한 필요

파일 생성 시에는 다음 두 가지가 필요하다.

1. 접근 모드(읽기/쓰기)
2. 파일 권한(permission)

Linux에서는 파일 권한을

- 사용자(user)
- 그룹(group)
- 기타 사용자(world)

로 구분한다.

이 예제에서는 사용자 권한만 다룬다.

[2] 시스템 호출 인수

Register	내용
rax	호출 코드 = SYS_creat (85)
rdi	NULL 종료 파일 이름 문자열 주소
rsi	파일 접근 모드 플래그

```
SYS_creat equ 85
O_CREAT   equ 0x40
O_TRUNC   equ 0x200
O_APPEND  equ 0x400

S_IRUSR   equ 00400q
S_IWUSR   equ 00200q
S_IXUSR   equ 00100q
```

[3] 권한 결합 예

S_IRUSR | S_IWUSR

의 의미

- 소유자 읽기 허용
- 소유자 쓰기 허용

|는 비트 OR 연산이다.

여러 권한을 동시에 활성화할 때 사용한다.

[4] 반환 값

- 성공: rax = 파일 디스크립터
- 실패: rax < 0

이 파일 디스크립터는 이후 모든 파일 작업(read, write, close)에서 사용된다.

[5] open vs open/create 핵심 차이

구분	open	open/create
파일 존재 필요	반드시 존재	없어도 됨
기존 파일 처리	그대로 열기	삭제 후 새로 생성
권한 지정 필요	X	O
반환 값	파일 디스크립터	파일 디스크립터

[13-6] 파일 읽기(File Read)

파일을 읽기 전에 적절한 파일 플래그로 파일을 먼저 열어야 한다.

파일 read 시스템 서비스의 인수는 다음과 같다.

Register	SYS_read
rax	호출 코드 = SYS_read (0)
rdi	파일 디스크립터 (열린 파일의)
rsi	읽은 문자를 저장할 주소
rdx	읽을 문자 수

다음과 같은 선언이 있다고 가정하자.

```
SYS_read equ 0 ; file read
```

파일 read 작업이 실패하면 rax 레지스터에 음수 값이 반환된다.

파일 read 작업이 성공하면 실제로 읽은 문자 수가 반환된다.

완전한 파일 읽기 예제는 다음 절을 참고하라.

[13-7] 파일 쓰기(File Write)

파일 write 시스템 서비스의 인수는 다음과 같다.

Register	SYS_write
rax	호출 코드 = SYS_write (1)
rdi	파일 디스크립터 (열린 파일의)
rsi	쓸 문자들의 주소
rdx	쓸 문자 수

다음과 같은 선언이 있다고 가정하자.

```
SYS_write equ 1; file write
```

파일 write 작업이 실패하면 rax 레지스터에 음수 값이 반환된다.

파일 write 작업이 성공하면 실제로 기록된 문자 수가 반환된다.

완전한 파일 쓰기 예제는 Example File Read 절을 참고하라.

[13-8] 파일 작업 예제(File Operations Examples)

이 절에는 기본적인 파일 I/O 작업을 보여주는 간단한 예제 프로그램들이 포함되어 있다.

파일 I/O 버퍼링과 관련된 보다 복잡한 문제는 다음 장에서 다룬다.

[13-8-1] 예제: 파일 쓰기

이 예제 프로그램은 짧은 메시지를 파일에 기록한다.

생성된 파일에는 간단한 메시지(이 예제에서는 URL)가 들어 있다.

파일 이름과 파일에 기록할 메시지는 하드코딩되어 있다.

이는 예제를 단순하게 만들기 위함이며 현실적인 방식은 아니다.

open/create 서비스를 사용하므로 파일이 생성된다. (기존 파일이 있다면 덮어쓴다.)

```
; Example program to demonstrate file I/O.
```

```
; This example will open/create a file,
```

```
; write some information to the file,
```

```
; and close the file.
```

```
; Note, the file name and write message
```

```
; are hard-coded for the example.
```

```
section .data
```

```
; -----
```

```
; Define standard constants.
```

```
LF equ 10
```

```
NULL equ 0
```

```
TRUE equ 1
FALSE equ 0
EXIT_SUCCESS equ 0
STDIN equ 0
STDOUT equ 1
STDERR equ 2
SYS_read equ 0
SYS_write equ 1
SYS_open equ 2
SYS_close equ 3
SYS_fork equ 57
SYS_exit equ 60
SYS_creat equ 85
SYS_time equ 201

O_CREAT equ 0x40
O_TRUNC equ 0x200
O_APPEND equ 0x400

O_RDONLY equ 000000q
O_WRONLY equ 000001q
O_RDWR equ 000002q

S_IRUSR equ 00400q
S_IWUSR equ 00200q
S_IXUSR equ 00100q

; -----
; Variables for main.

newLine db LF, NULL
header db LF, "File Write Example."
db LF, LF, NULL

fileName db "url.txt", NULL

url db "http://www.google.com"
db LF, NULL

len dq $-url-1
```

```
writeDone db "Write Completed.", LF, NULL

fileDesc dq 0

errMsgOpen db "Error opening file.", LF, NULL
errMsgWrite db "Error writing to file.", LF, NULL

section .text
global _start

_start:
; -----
; Display header line...

mov rdi, header
call printString

; -----
; Attempt to open file.

mov rax, SYS_creat
mov rdi, fileName
mov rsi, S_IRUSR | S_IWUSR
syscall

cmp rax, 0
jl errorOnOpen

mov qword [fileDesc], rax
; -----
; Write to file.

mov rax, SYS_write
mov rdi, qword [fileDesc]
mov rsi, url
mov rdx, qword [len]
syscall

cmp rax, 0
jl errorOnWrite
```



```
mov rdi, writeDone
call printString
; -----
; Close the file.

mov rax, SYS_close
mov rdi, qword [fileDesc]
syscall

jmp exampleDone
; -----
; Error on open.

errorOnOpen:
mov rdi, errMsgOpen
call printString
jmp exampleDone
; -----
; Error on write.

errorOnWrite:
mov rdi, errMsgWrite
call printString
jmp exampleDone
; -----
; Example program done.

exampleDone:
mov rax, SYS_exit
mov rdi, EXIT_SUCCESS
syscall

global printString
printString:

push rbp
mov rbp, rsp
push rbx

mov rbx, rdi
mov rdx, 0
```

```

strCountLoop:
    cmp byte [rbx], NULL
    je strCountDone
    inc rdx
    inc rbx
    jmp strCountLoop

strCountDone:
    cmp rdx, 0
    je prtDone

mov rax, SYS_write
mov rsi, rdi
mov rdi, STDOUT
syscall

prtDone:
pop rbx
pop rbp
ret

```

이 프로그램은 다음 작업을 수행한다.

1. 화면에 헤더 출력
2. 파일을 생성(open/create)
3. 문자열을 파일에 기록(write)
4. 파일을 닫음(close)
5. 프로그램 종료(exit)

[1] 상수 정의

```

SYS_read  equ 0
SYS_write equ 1
SYS_open  equ 2
SYS_close equ 3
SYS_exit  equ 60
SYS_creat equ 85

```

이 값들은 Linux86-64 시스템 콜 번호이다.

rax 에 이 값을 넣고 syscall 을 호출하면 커널이 해당 작업을 수행한다.

파일 관련 플래그

```
O_CREAT equ 0x40
O_TRUNC equ 0x200
O_APPEND equ 0x400
```

파일 생성 또는 동작 방식을 제어하는 비트 플래그이다.

접근 권한

```
O_RDONLY equ 000000q
O_WRONLY equ 000001q
O_RDWR equ 000002q
```

8 진수(octal)표기이며, Linux 파일 권한 체계와 동일하다.

파일 권한

```
S_IRUSR equ 00400q
S_IWUSR equ 00200q
S_IXUSR equ 00100q
```

- 읽기
- 쓰기
- 실행

여기서는:

```
S_IRUSR | S_IWUSR
```

- 파일 소유자에게 읽기/쓰기 허용

[2] 데이터 영역 설명

```
fileName db "url.txt", NULL
```

생성될 파일 이름

```
url db "http://www.google.com"
db LF, NULL
```

파일에 기록할 문자열

길이 계산

```
len dq $-url-1
```

- \$ = 현재 위치
- \$ - url = url 시작부터 현재까지의 바이트 수
- -1 = NULL 제외

즉, 실제 출력할 문자열 길이

[3] 프로그램 실행 흐름(_start)

[3-1] 헤더 출력

```
mov rdi, header  
call printString
```

- rdi = 문자열 주소
- printString 호출

[3-2] 파일 생성

```
mov rax, SYS_create  
mov rdi, fileName  
mov rsi, S_IRUSR | S_IWUSR  
syscall
```

레지스터 의미

레지스터	의미
rax	시스템콜 번호 (85)
rdi	파일 이름 주소
rsi	파일 권한

성공하면:

```
rax = 파일 디스크립터
```

실패하면

```
rax < 0
```

오류 검사

```
cmp rax, 0  
jl errorOnOpen
```

jl = jump if less than
즉, 음수이면 오류 처리

파일 디스크립터 저장

```
mov qword [fileDesc], rax
```

이후 write, close 에서 필요하기 때문

[4] 파일에 쓰기(SYS_write)

```
mov rax, SYS_write
mov rdi, qword [fileDesc]
mov rsi, url
mov rdx, qword [len]
syscall
```

레지스터 의미

레지스터	의미
rax	SYS_write
rdi	파일 디스크립터
rsi	버퍼 주소
rdx	바이트 수

성공하면

```
rax = 실제 기록된 바이트 수
```

실패하면:

```
rax < 0
```

오류 검사

```
cmp rax, 0
jl errorOnWrite
```

[5] 파일 닫기(SYS_close)

```
mov rax, SYS_close
mov rdi, qword [fileDesc]
syscall
```

파일 디스크립터는 반드시 닫아야 한다.

닫지 않으면

- 리소스 누수 발생
- 파일 테이블이 가득 찰 수 있음

[6] 프로그램 종료

```
mov rax, SYS_exit
mov rdi, EXIT_SUCCESS
syscall
```

정상 종료 코드 0 반환

[7] printString 함수 분석

이 함수는 NULL 종료 문자열 출력 함수이다.

[7-1] 함수 프로로그

```
push rbp
mov rbp, rsp
push rbx
```

- rbx 는 callee-saved 레지스터
- 사용하기 때문에 반드시 복구해야 함

[7-2] 문자열 길이 계산

```
mov rbx, rdi
mov rdx, 0
```

rbx = 문자열 주소
rdx = 카운터

루프:

```
cmp byte [rbx], NULL
je strCountDone
inc rdx
inc rbx
```

- NULL 전까지 길이 계산

[7-3] write 호출

```
mov rax, SYS_write
mov rsi, rdi
mov rdi, STDOUT
syscall
```

STDOUT(1)에 출력

[7-4] 함수 에필로그

```
pop rbx
pop rbp
ret
```

rbx 복원 (callee-saved 규칙 준수)

[8] 전체 구조 요약

```
_start
├─ printString (헤더 출력)
├─ SYS_creat (파일 생성)
├─ SYS_write (파일 기록)
├─ SYS_close (파일 닫기)
└─ SYS_exit (종료)
```

[13-8-1] 예제: 파일 읽기

이 예제는 파일을 읽는다.

읽을 파일에는 이전 예제의 URL 과 같은 간단한 메시지가 들어 있다.

파일 이름은 하드코딩되어 있으며 이는 예제를 단순하게 만들기 위함이지만 현실적인 방식은 아니다.

사용되는 파일 이름은 이전 파일 쓰기 예제와 동일하다.

이 예제 프로그램을 파일 쓰기 예제 프로그램보다 먼저 실행하면 파일이 존재하지 않으므로 오류가 발생한다.

파일 쓰기 예제 프로그램을 실행한 후에 이 파일 읽기 예제 프로그램을 실행하면 파일을 읽고 그 내용을 화면에 출력한다.

```
; Example program to demonstrate file I/O.
; This example program will open a file,
; read the contents, and write the contents to the screen.
; This routine also provides some very simple examples
; regarding handling various errors on system services.
; Note, the file name is hard-coded for this example.
; -----

section .data
; -----
; Define standard constants.

LF equ 10
NULL equ 0
TRUE equ 1
FALSE equ 0
EXIT_SUCCESS equ 0
STDIN equ 0
STDOUT equ 1
STDERR equ 2
```

```
SYS_read equ 0
SYS_write equ 1
SYS_open equ 2
SYS_close equ 3
SYS_fork equ 57
SYS_exit equ 60
SYS_creat equ 85
SYS_time equ 201

O_CREAT equ 0x40
O_TRUNC equ 0x200
O_APPEND equ 0x400

O_RDONLY equ 000000q
O_WRONLY equ 000001q
O_RDWR equ 000002q

S_IRUSR equ 00400q
S_IWUSR equ 00200q
S_IXUSR equ 00100q
; -----
; Variables/constants for main.

BUFF_SIZE equ 255

newLine db LF, NULL

header db LF, "File Read Example."
db LF, LF, NULL

fileName db "url.txt", NULL

fileDesc dq 0

errMsgOpen db "Error opening the file.", LF, NULL
errMsgRead db "Error reading from the file.", LF, NULL

section .bss
readBuffer resb BUFF_SIZE
section .text
global _start
```



```

_start:
; -----
; Display header line...

mov rdi, header
call printString

; -----
; Attempt to open file – Use system service for file open
mov rax, SYS_open
mov rdi, fileName
mov rsi, O_RDONLY
syscall

cmp rax, 0
jl errorOnOpen

mov qword [fileDesc], rax
; -----
; Read from file.

mov rax, SYS_read
mov rdi, qword [fileDesc]
mov rsi, readBuffer
mov rdx, BUFF_SIZE
syscall

cmp rax, 0
jl errorOnRead

; -----
; Print the buffer.
; add the NULL for the print string

mov rsi, readBuffer
mov byte [rsi+rax], NULL
mov rdi, readBuffer
call printString
printNewLine

```

```
; -----  
; Close the file.  
  
mov rax, SYS_close  
mov rdi, qword [fileDesc]  
syscall  
  
jmp exampleDone  
  
; -----  
; Error on open.  
  
errorOnOpen:  
mov rdi, errMsgOpen  
call printString  
jmp exampleDone  
  
; -----  
; Error on read.  
  
errorOnRead:  
mov rdi, errMsgRead  
call printString  
jmp exampleDone  
  
; -----  
; Example program done.  
  
exampleDone:  
mov rax, SYS_exit  
mov rdi, EXIT_SUCCESS  
syscall  
  
global printString  
printString:  
  
push rbp  
mov rbp, rsp  
push rbx  
  
mov rbx, rdi
```

```
mov rdx, 0

strCountLoop:
    cmp byte [rbx], NULL
    je strCountDone
    inc rdx
    inc rbx
    jmp strCountLoop

strCountDone:
    cmp rdx, 0
    je prtDone

mov rax, SYS_write
mov rsi, rdi
mov rdi, STDOUT
syscall

prtDone:
pop rbx
pop rbp
ret
```

printStrng()함수는 두 예제에서 완전히 동일하며 각 프로그램이 독립적으로 어셈블 및 실행될 수 있도록 각가에 반복 포함되어 있다.